Neural means and kernel corrections for operator learning

Yitzchak Shmalo YITZCHAK.SHMALO@GMAIL.COM
Einstein Institute of Mathematics
The Hebrew University of Jerusalem
Jerusalem, Israel

Abstract

Where should an exact kernel regression sit relative to a trained neural network? We study the question on two operator-learning problems, replicating every stage at ten seeds. On the structural-mechanics benchmark of de Hoop et al., where kernels and neural operators are competitive, the kernel corrects the residual of a stacked neural mean: the pipeline reaches $4.546 \pm 0.003\%$ test error, within the test block's sampling error of the best published method (PARA-Net, 4.55%) and below every other, and $5.433 \pm 0.093\%$ in the 1250-sample protocol against a published 6.49%. On the OCO-2 radiative-transfer problem of Lamminpää et al., where the same kernel on the raw state trails the network tenfold, it belongs on the network's learned features, and a per-coordinate combination improves on the published Gaussian-process emulator on that paper's two rescored error metrics on all three bands. The replicates show why ensembles saturate on structural mechanics: every architecture's residuals align above 0.82 at every seed, sixty predictors reach 4.581% at hindsight-optimal convex weights, and a floor theorem for pools of seeds and architectures places that number, for the pool trained, from three measured alignments. The supporting bounds are stated with their conditions and checked numerically.

Keywords: operator learning, kernel ridge regression, residual correction, ensembles, conformal prediction

1 Introduction

Operator learning constructs fast surrogates for the solution maps of parametric partial differential equations and physical forward models.[1] Neural operators (Kovachki et al., 2023; Lu et al., 2021; Li et al., 2021) learn their own representations; kernel and Gaussian-process methods (Batlle et al., 2024; Mora et al., 2025; Owhadi and Scovel, 2019) come with exact solves and error bounds and a strong record on the same benchmarks. Placing a neural network in the mean of a kernel or Gaussian-process regression is an established way of combining the two (Mora et al., 2025), and so is putting a kernel on a network's learned features (Wilson et al., 2016; Calandra et al., 2016). This paper takes a plain version of that pairing — a neural mean, trained in the metric the application reports, followed by an exact Matérn kernel regression fitted after the network — and asks, on two public problems with published baselines, what the kernel stage is for. The answer differs between the two problems, and the difference is the subject of the paper. Table 1 collects the published numbers and ours.

On the structural-mechanics benchmark of de Hoop et al. (2022) the two families are competitive: the published neural operators and the kernel method of Batlle et al. (2024) sit

1. Code, data pointers, and the per-run summaries behind every number reported here are at `https://github.com/yspennstate/neural-means-kernel-corrections`.

Table 1: The two problems studied in depth. OCO-2 rows compare against the Gaussian-process emulator of Lamminpää et al. (2025), scored from its own stored predictions on the same 2000 test points, in both of that paper's error metrics (Section 5); ours are mean ± standard deviation over ten seeds per band. Structural-mechanics rows quote the best published mean relative L^2 test error at each training size (de Hoop et al., 2022; Mora et al., 2025), reported without uncertainties in the original work; ours are mean ± standard deviation over ten seeds, and the 20000-sample comparison is a tie within the test block's own sampling error (Section 4.4).

Problem	Metric	Published	This work
OCO-2, O2 band	reduced	16.89%	$4.12 \pm 0.05\%$
	radiance	0.0448%	$0.0294 \pm 0.0020\%$
OCO-2, weak CO_2	reduced	24.06%	$16.28 \pm 0.06\%$
	radiance	0.0599%	$0.0507 \pm 0.0052\%$
OCO-2, strong CO_2	reduced	16.14%	$8.08 \pm 0.01\%$
	radiance	0.1147%	$0.0584 \pm 0.0041\%$
Structural mechanics, 20000 samples	relative L^2	4.55% (PARA-Net)	$4.546 \pm 0.003\%$
Structural mechanics, 1250 samples	relative L^2	6.49% (FNO-mean GP)	$5.433 \pm 0.093\%$

within 0.65 points of one another, and our own kernel ridge reproduces the published kernel number. There the kernel's role is to correct the residual of a stacked neural mean, and the corrected pipeline reaches $4.546 \pm 0.003\%$ under the 20000-sample protocol, replicated at ten independently seeded runs: within the test block's own sampling error of the best published number, PARA-Net's 4.55%, which is a single run reported without an uncertainty and whose predictions are not available for a paired comparison, and below every other. We call that a tie, and Section 4.4 says exactly what the word covers. Under the 1250-sample protocol of Mora et al. (2025) it reaches $5.433 \pm 0.093\%$ against a published best of 6.49%. The published numbers on this benchmark carry no uncertainties, and replication is what lets gains of a few hundredths of a point be resolved at all; it also shows why ensembles saturate there. The residuals of every architecture we train correlate between 0.82 and 0.98 at every seed; their shared component concentrates at the corners of the loaded edge, where the finite element solution is least reliable, and moves by hundredths with ensemble diversity and with training-set size; and the spread of our own pipelines across seeds, 0.010 and 0.003 points, is sixty to two hundred times smaller than the span of the published architectures, so the published cluster near 4.5% is not run-to-run noise. That evidence is consistent with a floor set by the benchmark's data rather than by any surrogate class, but it is circumstantial: the experiment that would decide it — re-solving a set of the hardest cases on a refined mesh or with higher-order elements and comparing the solver discrepancy with the shared residual — has not been run, and Section 4.5 says what each measurement does and does not establish.

On the OCO-2 radiative-transfer emulation problem of Lamminpää et al. (2025) the same components separate by an order of magnitude, and the kernel's role changes. Per spectral band of the satellite, the task maps a reduced atmospheric state to a reduced radiance

spectrum, and the baseline is that paper's kernel-flow emulator, whose stored predictions we score on the same test points. An exact Matérn regression on the raw state reaches 40% on the reduced metric and the network 4.4%; the same regression on the trained network's features reaches 4.1%, past the network it feeds on, at ten of ten seeds on each band. The comparison is controlled — the same kernel, tuning routine and budget on both inputs — and it locates the difference: the effective dimension of the two Gram matrices is essentially the same, the design factor measured at the test points falls by about half, and the squared native-space norm of the kernel interpolant of the targets — the computable lower bound on the target's own norm — falls by a factor of about forty at the matched setting. Read as finite-design diagnostics, and with their dependence on the kernel scale reported, these are the direction the pullback identity of Proposition 6.8 predicts for a learned feature map and a rescaling of the input cannot produce. Combined per output coordinate, across networks trained in the flat and in metric-weighted losses and the kernel heads on their features, the surrogate improves on the emulator on both of that problem's error metrics on all three bands, at ten seeds per band; one band's radiance margin is thin, and Section 5 gives the per-seed counts.

What this paper adds to the prior constructions is therefore not the pairing but what it is used to measure: an exact post-hoc residual correction at $n = 19000$, one dense Cholesky factorization rather than an approximation; the controlled raw-input versus learned-feature comparison above; the per-coordinate treatment of OCO-2's two metrics, which trade off coordinate by coordinate and had been won by different models; the residual-correlation diagnosis of why ensembles saturate on the structural-mechanics benchmark, carried to a pool of sixty predictors; and a replicated audit, at ten seeds, of every stage. The theory of Section 6 is a toolkit for reading those measurements, kept in proportion and stated with its conditions: a symmetrization inequality behind the reflection averaging; a second-moment identity that predicts the stacked risk from measured residual correlations, brackets how close any convex ensemble sits to its floor, and, for pools of seeds and architectures, separates what reseeding can buy from what a new architecture can (Theorem 6.6); capacity bounds for the fitted stacks and the grid-tuned correction, with constants measured on the runs, read as conditional capacity calculations rather than guarantees for the shipped estimator, since its selections reuse the validation split; the conditional power-function bound of Theorem 6.9, sharp over the native-space ball and, by Theorem 6.10, minimax there, and the pullback identity; an informal rate calculation (Proposition S2.5) for what an input metric can buy a kernel, whose rate half the OCO-2 sweep does not bear out; and a distribution-free coverage statement whose hypotheses the calibration protocol satisfies. The last matters because the design factor in Theorem 6.9, though a valid bound, ranks the pointwise error poorly on the benchmark; its split-conformal rescaling holds its nominal coverage at every seed, and so does a constant-width band calibrated the same way, so what the design factor contributes on this benchmark is the guarantee rather than the ranking (Section 4.7). Each statement is checked numerically (Supplement S10); the two-member stacking rule is scored prospectively on every member pair of every seed and reported as it came out, reliable outside a margin band set by its own estimation noise and no better than chance inside it (Section 5.2).

Section 2 describes the problems, protocols, published results and related work, and includes a survey of the same recipe on the remaining problems of the operator-learning suite, which is uncontrolled and reported as such. Section 3 specifies the pipeline. Sections 4 and 5

report the two studies. Section 6 states the supporting theory, with proofs in Supplement S8 and their numerical verification in Supplement S10, and Section 7 discusses limitations.

2 Benchmark, protocol, and related work

2.1 Problem and data

The dataset originates in the cost-accuracy study of de Hoop et al. (2022) and is the one distributed with Batlle et al. (2024) and Mora et al. (2025). An isotropic elastic plate occupies $D = (0,1)^2$; the displacement field w satisfies $\nabla \cdot \sigma = 0$ with a fixed constitutive law, displacement conditions on the bottom and lateral parts of the boundary, and a prescribed normal traction $\bar{t}$ on the top edge $\Gamma_t = [0,1] \times \{1\}$. The quantity of interest is the von Mises stress field σ_v on D. The learning task is the map $G : \bar{t} \mapsto \sigma_v$. Loads are drawn from the Gaussian field $\mathcal{GP}\big(100,\, 400^2(-\Delta+3^2 I)^{-1}\big)$ with homogeneous Neumann boundary conditions for the Laplacian; outputs are finite element solutions interpolated to a regular 41×41 grid, and the load is sampled at 41 points. The distributed file contains 40000 input/output pairs; the input array stores the load broadcast along the second grid coordinate, which we verified is an exact copy (maximal deviation 0 across all 40000 samples), so all our methods consume the load as a vector in $\mathbb{R}^{41}$.

Errors are mean relative L^2 errors over the test set,

$$\frac{1}{N_{\text{test}}} \sum_{n} \frac{\|\widehat{G}(u_n) - G(u_n)\|_2}{\|G(u_n)\|_2},$$

computed on grid values in double precision. Quadrature weighting (trapezoidal instead of plain vector norms) changes our reported numbers by less than 0.02 percentage points, and we report the plain-norm convention of the prior work.

2.2 Protocols and published results

Two regimes appear in the literature, and we follow both. In the *high-data* protocol the first 20000 samples are available for training and the last 20000 form the test set; de Hoop et al. (2022) report, at 20000 training samples, 4.55% (PARA-Net), 4.67% (PCA-Net), 4.76% (FNO) and 5.20% (DeepONet), and Batlle et al. (2024) report 5.18% for their optimal-recovery kernel method (Matérn/rational quadratic; 27.11% for the linear kernel) on the same task. Within the 20000-sample budget we hold out the last 1000 samples (of a fixed permutation) for model selection and stacking, and train on the remaining 19000, so no method of ours sees more than the 20000 training samples used by the baselines. In the *low-data* protocol of Mora et al. (2025), 1250 samples are available and the same 20000-sample test set is used; their table gives 8.70% (DeepONet), 6.62% (FNO), 6.95% (the kernel method of Batlle et al. 2024), 6.74% (their zero-mean GP), 7.12% (DeepONet-mean GP) and 6.49% (FNO-mean GP). In this regime we carve the validation split (250 samples) out of the 1250, so training uses 1000 samples and every choice made by the pipeline is informed by the 1250 available labels only.

2.3 The rest of the suite, and a check of the mirror symmetry

Two observations belong to the setting rather than to the results, and both are reported in Supplement S4. The same fixed recipe was run once, unchanged, on the other problems of the operator-learning suite that Batlle et al. (2024) assembled from de Hoop et al. (2022) and Lu et al. (2022). That survey is uncontrolled — single runs, no seed replication, and in two cases training splits or grids that differ from the baselines' — so nothing is drawn from it beyond orientation: where the map is smooth and data are plentiful the kernel stage wins the recipe's own selection and lands within a factor of two of the tuned per-problem kernels. The two problems studied in depth, with replication, are the subject of the rest of the paper. Separately, the mirror symmetry that Proposition S1.1 assumes is tested on the data rather than taken on faith: outputs of near-mirror inputs are some three times closer to one another than the inputs are, the correct reflection axis is singled out against the alternative, and the empirical mean and covariance of the training loads are reflection-invariant to within 1.1% and 1.5%.

2.4 Related work

The benchmark sits at the meeting point of three lines of work. *Neural operators* learn maps between function spaces: DeepONet (Lu et al., 2021) with its branch/trunk factorization, the Fourier neural operator (Li et al., 2021) and the broader neural-operator framework (Kovachki et al., 2023), and the reduced-basis PCA-Net and PARA-Net architectures assembled for the cost–accuracy study of de Hoop et al. (2022). These methods are fast and mesh-flexible but do not by themselves come with error control, and it is their numbers on this problem that define the plateau we start from. *Kernel and Gaussian process methods* for operator learning approach the same maps through reproducing kernels: the optimal-recovery framework of Batlle et al. (2024), with its convergence theory and a-priori bounds, is the most direct comparison, and it is competitive with neural operators on most of the benchmarks it considers. Data-adapted kernels learned by cross-validation (Darcy et al., 2023) and vector-valued kernel formulations extend the reach of this family. *Hybrids* that place a neural network and a kernel in the same estimator are the closest relatives of our pipeline: Mora et al. (2025) use a neural operator as the mean of a Gaussian process and fit the two together, and this is the work we most directly build on, differing in that we fit the mean first and the kernel after, tune by cross-validation rather than marginal likelihood, and solve the correction exactly at nineteen thousand points. Kernels on learned representations are likewise established: deep kernel learning (Wilson et al., 2016; Calandra et al., 2016) places a Gaussian process on the features of a network and trains through it, and the recent REEF-GP of Vendrell-Gallart et al. (2026) places a Gaussian process on the residuals of a trained neural operator in its learned feature space, post hoc, for uncertainty quantification. Our feature-space stage is the post-hoc form of the same construction, with the exact solve and validation tuning used everywhere else in the pipeline; it differs from REEF-GP in what it is used for and how it is judged — as a predictor, scored against the same kernel on the raw input under one tuning routine and budget (Section 5), with coverage supplied by split-conformal calibration rather than by the posterior — and what we add is not the construction but that controlled comparison. The residual construction itself is older than any of these: it is the discrepancy model of Kennedy and O'Hagan (2000), in which

a Gaussian process corrects a cheap approximation, and the reduced-order-model error surrogates of Drohmann and Carlberg (2015) put a Gaussian process on the error of a physics-based surrogate in the same way; universal kriging with a trained mean is the same estimator in geostatistical dress. Within operator learning, Kumar et al. (2024) induce a Gaussian process from a neural operator, Magnani et al. (2025) linearize a trained neural operator into a function-valued Gaussian process, and Ma et al. (2024) calibrate operator predictions by split conformal prediction, the same coverage device we use in Section 4.7. Ober et al. (2021) document how deep kernel learning overfits when the feature map and the kernel are trained together and the same data select both, which is one reason our feature-space stage trains the network first and tunes the kernel by validation afterwards.

Two further threads inform the design. The kernel-flow method (Owhadi and Yoo, 2019) learns a kernel from data by minimizing a half-sample cross-validation loss, and its use as a regularizer for the inner layers of a network (Yoo and Owhadi, 2021) is exactly the term we analyze in Proposition S1.2; kernel flows have since been used at scale as emulators for physical forward models, for instance in atmospheric radiative transfer retrievals (Lamminpää et al., 2025) and in the inference of convective-storm structure from satellite observations (Prasanth et al., 2021), where presenting the input in its physical parametrization and adapting the kernel to data are what make the emulator accurate. Our pipeline follows the same instinct, with the smoothing of the target performed by a neural ensemble rather than by a hand-chosen reduction. Finally, the effective-dimension reading of the kernel solve (Lemma S5.1) draws on the random-matrix description of kernel Gram spectra (Koltchinskii and Giné, 2000) and on the classical Marčenko–Pastur (Marčenko and Pastur, 1967) and spiked-covariance (Baik et al., 2005) laws; the same effective dimension governs the generalization of kernel ridge regression (Caponnetto and De Vito, 2007).

3 Method

The pipeline has three stages: neural means, stacking, and kernel corrections. All components operate on the load vector $u \in \mathbb{R}^{41}$ (standardized by training statistics) and produce stress fields in $\mathbb{R}^{41\times 41}$; all networks are trained with the reported metric as the loss, i.e. the per-sample relative L^2 error in original units, which we found mildly but consistently better than mean squared error on normalized targets.

3.1 Neural means

Residual MLP. The primary mean is a residual multilayer perceptron: three or five residual SiLU blocks of width 1024–1536 mapping $\mathbb{R}^{41} \to \mathbb{R}^{1681}$ directly. This is essentially PARA-Net (de Hoop et al., 2022) with a modern training recipe, and it also supplies the penultimate features used by the feature-space kernel stage.

Kernel-conditioned refiner. The second mean is the same residual network given an extra input channel: the kernel method's predicted stress field for the same load, concatenated with the load. It therefore learns to correct the kernel predictor rather than to reproduce the map from scratch; on the metric it sits within two hundredths of a point of the best single members (the normalized-MSE network and the complete-schedule FNO, Table 4). During training the refiner reads *out-of-fold* kernel predictions (four-fold, so the kernel channel

is never fit on the target sample), and at test time the full-data kernel prediction, so the channel never sees its own target.

Other instances. The mean slot is not tied to a particular architecture. We also use a Fourier neural operator (Li et al., 2021) that consumes the broadcast load as a field and an MSE-trained variant of the MLP; both are drop-in means, both enter the ensemble of Section 4.5, and their pairwise error correlations are one of the measurements of this paper. A UNet on the same field representation is the sixth member of the second campaign (Section 4.1). We additionally implement an encoder–decoder transformer (Vaswani et al., 2017; Dosovitskiy et al., 2021) that tokenizes the 41 load samples and decodes the field by cross-attention at the grid points; it is a natural further architecture family, but we were unable to train it to the level of the others on the hardware available for this study and report the ensemble without it (Supplement S9).

All means are trained with AdamW and a cosine schedule, batches of 128–256, for up to a few hundred epochs (Supplement S9 lists every hyperparameter). During training each batch is reflected with probability 1/2 (load reversed, target field reflected along the first grid coordinate); at test time each model is evaluated as $\frac{1}{2}(f(u)+Tf(Su))$. Proposition S1.1 shows the test-time step cannot hurt on average, and the ablation confirms small consistent gains from both steps. Optionally, the training loss carries a kernel-flow term $\beta\, e_2$ computed from the batch (Section S1.2) with a Gaussian kernel on pooled penultimate features whose log-bandwidth is trained jointly; this variant appears in the ablation.

3.2 Stacking

With M trained models $f_1, \dots, f_M$ we form the convex combination $m(u) = \sum_m w_m f_m(u)$, $w \in \Delta^{M-1}$, minimizing the validation relative error; the weights are initialized at the simplex minimizer of the measured second-moment matrix of Proposition 6.1 and polished by a short search on the 1000 held-out samples (250 in the low-data protocol). A per-pixel variant allows the weights (plus an intercept) to vary over the grid, ridge-fitted on half the validation split and accepted only when it beats the global weights on the other half; it is the variant the final pipeline uses. Stacking on a held-out split rather than uniform averaging costs nothing, guards against a weak member (Wolpert, 1992), and is statistically almost free at this size (Proposition S1.3).

3.3 Kernel corrections

The stacked mean is then corrected by kernel ridge regression of its residuals. Let $X = (u_1, \dots, u_n)$ be the training loads (standardized coordinatewise), $R \in \mathbb{R}^{n\times 1681}$ the matrix of training residuals $v_i - m(u_i)$, and k a Matérn-5/2 kernel $k(u,u') = \kappa\big(\|u-u'\|/s\big)$. The correction is

$$\widehat{r}(u) = k(u, X)\,\alpha, \qquad \alpha = (K + n\lambda I)^{-1} R,$$

with (s, λ) chosen on the validation split from a small grid (s as a multiple of the median pairwise distance, $\lambda \in \{10^{-6}, 10^{-5}, 10^{-3}\}$; the grid is searched on an 8000-sample subsample and the chosen pair is refit on all of X). The corrected surrogate is $m+\widehat{r}$. A second corrective stage repeats the construction with the kernel evaluated on deep features, the concatenated penultimate activations of the ensemble members (reflection-averaged, standardized), fitted to the residuals of $m + \widehat{r}$; it contributes a smaller improvement and is included when it

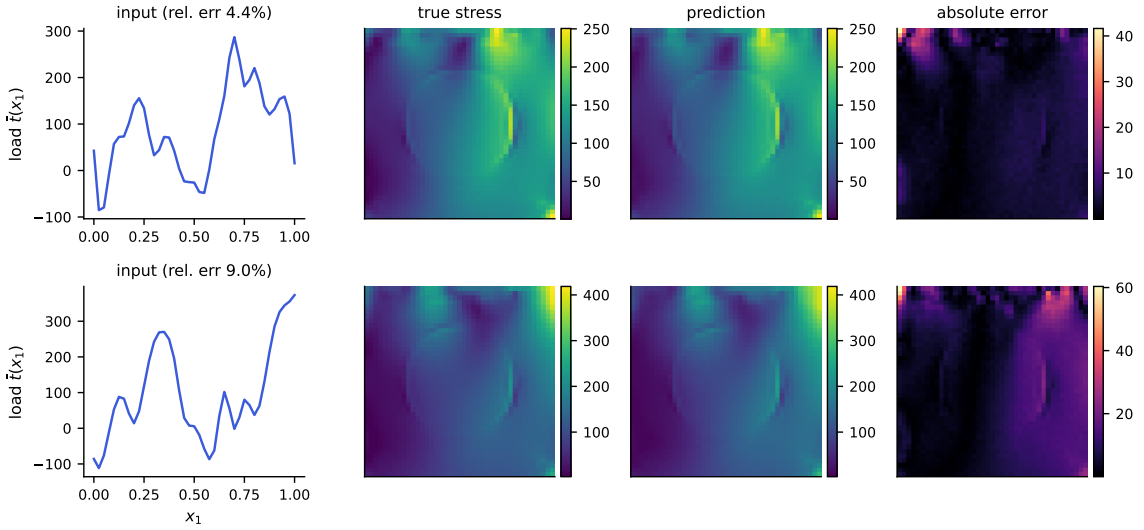

Figure 1: Example predictions of the full pipeline. Top: a median-error test case; bottom: a case at the 98th error percentile. Columns: input load, true von Mises stress, prediction, and pointwise absolute error (note the smaller scale). Errors concentrate in the high-traction corner, as also reported by Mora et al. (2025).

helps on validation. At $n = 19000$ the exact solve is a single Cholesky factorization of a 19000×19000 matrix, under half a minute in double precision on a laptop CPU (measured in Section S6.4), and prediction is two matrix products; no inducing points, preconditioners, or stochastic solvers are involved. The Gaussian process reading of the same formulas supplies the posterior standard deviation $P_\lambda(u)$ of (1) at negligible extra cost (one triangular solve per evaluation batch), which is the design factor in the residual bound of Theorem 6.9 and the uncertainty signal studied in Section 4.7.

Two remarks on scope. Nothing in the construction uses properties of this particular PDE beyond the verified reflection symmetry, so the recipe (native input parametrization, symmetrized accurate means, validated kernel correction of the residual, a conformal band built on the posterior standard deviation) transfers to other operator learning problems with low-dimensional input parametrizations. And the pipeline degrades gracefully: dropping the feature stage, the stacking, or the corrections recovers progressively simpler methods whose individual numbers appear in the ablation table.

4 Results on the structural-mechanics benchmark

4.1 What is replicated, and at how many seeds

Every structural-mechanics number in this section is reported over ten seeds. A seed here is not a reinitialization of one network: it redraws the 1000-sample validation split from the training pool and reinitializes every trainable member, so the seed spread contains both the model-selection noise and the optimization noise. The test block of 20000 samples is fixed by the benchmark's convention and never moves, at any seed.

Two campaigns share those seeds. The first trains five members at every seed: the kernel ridge on loads, the residual MLP trained on the reported metric, the same architecture trained on normalized mean squared error, the kernel-conditioned refiner, and a Fourier neural operator. Its FNO member is the best-validation checkpoint of a run that a 36-hour task limit stopped between epochs 10 and 70 of a 200-epoch cosine schedule, so that member never annealed. The stack, the residual correction, the second-moment measurement and the conformal calibration are built on those five, and the four-member versions of each are kept, because the FNO's admission is a measurement and the reader should see what it was measured against. The second campaign, on one 40-core host, retrains the kernel ridge and the three MLP-family members under the same seeds — they reproduce the first campaign's test errors to the printed precision, seeded CPU training being deterministic — trains the FNO to the end of a 100-epoch cosine schedule, and adds a sixth member, a UNet on the same field representation. Every stage is then run again on the six. The first campaign is the reference for the stage-by-stage accounting of Sections 4.3–4.5; the second supplies the configuration this paper reports as its best, and repeats that accounting on six members.

The five-member set is not a configuration searched for after seeing results: it is the pre-specified six-member configuration minus the member the first campaign's task limit did not reach. Whether five members are preferred to four is decided on validation, per seed, by the same rule that governs every other stage choice here, and it prefers five at ten seeds out of ten (Section 4.3).

The low-data pipeline and the OCO-2 combination have the seed coverage stated where they appear, which is ten in both.

4.2 Main comparison

Table 2 reports the high-data protocol. The published numbers are quoted from de Hoop et al. (2022) (as tabulated by Batlle et al. 2024) and from Batlle et al. (2024); our rows are computed under the split of Section 2, which gives our methods strictly no more training information than the baselines had. The kernel ridge row reproduces the published kernel result almost exactly (5.197% against 5.18%, and it is the most reproducible row in the table: its seed standard deviation is 0.003 points, three thousandths of a point, because the only randomness it carries is the validation draw). We take that agreement as evidence that the protocols are aligned, so the gap opened by the rest of the pipeline is not an artifact of evaluation choices.

The five-member pipeline reaches $4.572 \pm 0.010\%$, the uncertainty being the spread over seeds, and the six-member pipeline of the second campaign $4.546 \pm 0.003\%$; the two are paired within seed, and the second is lower at every one of the ten, by 0.026 ± 0.010 points. Both are below every published method except PARA-Net, and neither is separable from PARA-Net: the differences are 0.022 and 0.004 points, against a test-block sampling term of 0.020 points computed from our own per-case errors (Section 4.4). We claim a tie there and nothing more. Against PCA-Net (4.67%) the differences are 0.098 and 0.124 points, and the margins over the FNO (4.76%), DeepONet (5.20%) and the optimal-recovery kernel (5.18%) are larger still; these are benchmark-score comparisons, not significance tests, because the published figures are single runs whose training variability and per-case errors are not available, so no paired or unpaired standard error for the difference can be

Table 2: Structural mechanics, high-data protocol (20000 training samples, 20000 test). Mean relative L^2 test error. Rows above the rule are published results on the same data, quoted without uncertainties because none were reported. Rows below are ours: mean ± standard deviation over ten seeds. The four-member row is the same pipeline with the FNO removed, kept because the FNO's admission is a measurement rather than a design choice (Section 4.3); the last two rows are the second campaign of Section 4.1, with the FNO trained to the end of its schedule and a UNet added. TTA denotes reflection averaging at test time.

Method	Error (%)	Parameters
DeepONet (de Hoop et al., 2022)	5.20	–
FNO (de Hoop et al., 2022)	4.76	–
PCA-Net (de Hoop et al., 2022)	4.67	–
PARA-Net (de Hoop et al., 2022)	4.55	–
Optimal-recovery kernel (Batlle et al., 2024)	5.18	–
Kernel ridge on loads (ours)	5.197 ± 0.003	–
Residual MLP + reflection TTA	4.836 ± 0.018	4.9M
+ affine stack over five members	4.595 ± 0.008	22.9M
+ residual kernel correction	4.572 ± 0.010	22.9M
the same pipeline without the FNO	4.611 ± 0.005	16.5M
+ complete FNO schedule and UNet, affine stack over six	4.567 ± 0.004	27.3M
+ residual kernel correction	4.546 ± 0.003	27.3M

formed. Without the spectral member the pipeline gives $4.611 \pm 0.005\%$, above PARA-Net by three times our own sampling term, so the tie belongs to the configurations that carry that member. Section 4.5 presents evidence, consistent with a floor in the benchmark data rather than a coincidence of tuning, for why the published cluster sits where it does, and says what would settle it.

4.3 Members and the stack, across seeds

Table 4 gives the five members of the first campaign and the stages built on them, then the second campaign's six. Four features of the first block are worth naming; the second block repeats the lesson of each.

The ordering of the members is stable at the top and not below it, and the seeds are what separate the two cases. Scored on the reflection-averaged validation predictions that the stack consumes, the normalized-MSE network is the most accurate single member at all ten seeds — not nine, ten. Second place is not stable: the refiner takes it at six seeds and the FNO at four, and on test the two are separated by 0.024 points against the FNO's own seed spread of 0.031. So the first rank survives ten independent redraws of the validation split and ten reinitializations and is not a coincidence of tuning; the second is a coin toss that a single-seed table would have reported as a fact. That contrast is the argument for the seeds, in one measurement.

Table 3: Low-data protocol (1250 training samples, 20000 test). Published rows from Mora et al. (2025). Our pipeline sees only the 1250 labels (250 of them used as the validation split). The pipeline row is mean ± standard deviation over ten seeds. Which stage the validation split selects varies with the seed — the input-space kernel correction at six seeds, its two-stage form at three, the feature-space one at the remaining seed — so the row is the error of the selected pipeline, not of a fixed final stage. The kernel-ridge row is the multiscale construction of Supplement S9, which has no random component under this protocol and therefore no seed spread.

Method	Error (%)
DeepONet	8.70
GP, optimal recovery (Batlle et al., 2024)	6.95
Zero-mean GP (Mora et al., 2025)	6.74
FNO	6.62
FNO-mean GP (Mora et al., 2025)	6.49
Kernel ridge on loads (ours, multiscale)	6.66
Full pipeline (ours, ten seeds)	5.433 ± 0.093

The FNO is the interesting member, and not because it is accurate. At $4.754 \pm 0.031\%$ it is third on average, behind both residual-MLP variants, though it passes the refiner at three seeds of the ten; its seed spread is three to five times theirs — a consequence of checkpoints early-stopped anywhere between epoch 10 and 70 (Supplement S9). Yet removing it costs the finished pipeline 0.039 ± 0.011 points, and validation prefers keeping it at ten seeds out of ten. Accuracy is not what a member contributes to a stack; the second-moment analysis of Section 4.5 says what is, and the simplex weights there give the FNO the largest single share despite its being third on the metric.

The second campaign says what the anneal was worth. Trained to the end of a 100-epoch schedule, the same architecture at the same seeds scores $4.682 \pm 0.009\%$, better than its early-stopped checkpoint at every seed by 0.072 ± 0.031 points, and its seed spread falls from 0.031 to 0.009 points: the spread was the checkpoint lottery, not the architecture. It is then the most accurate member at nine seeds of ten. The better member does not make a better stack. Rebuilt on the same five members with the annealed FNO in place of the early-stopped one, the corrected pipeline is $4.578 \pm 0.005\%$ against $4.572 \pm 0.010\%$, a paired difference of 0.006 ± 0.013 points in the wrong direction. The reason is in the residuals: the annealed FNO correlates 0.974–0.980 with the normalized-MSE network where the early-stopped one correlated 0.95–0.97, so what the member gained in accuracy it gave back in diversity, and the second-moment matrix predicts the wash before any stack is fit, the predicted best-mixture risk moving from 4.940% to 4.940% (Section 4.5). The six-member pipeline then sits 0.026 ± 0.010 points below the first campaign's, paired within seed, at ten of ten, and that gain is the UNet's: removing it from the six costs the corrected pipeline 0.032 ± 0.004 points at ten seeds of ten. The UNet lands fifth of six on the metric at $4.740 \pm 0.030\%$, behind both residual MLPs, and takes the largest convex share at four seeds of ten all the same. The second campaign repeats, with a new architecture, what the first

Table 4: Members and stages under the high-data protocol, mean ± standard deviation over ten seeds. Member errors are reflection-averaged where the member has a reflection; the kernel ridge and the refiner train with the symmetrization already applied. "Fitted" weights are solved on the validation split and read out once on test; equal weights involve no fitting. The lower block is the second campaign of Section 4.1; its kernel ridge and MLP-family members coincide with the upper block's to the printed precision and are not repeated.

Member or stage	Family / loss	Test error (%)
Kernel ridge on loads	Matérn-5/2	5.197 ± 0.003
Residual MLP	MLP, metric loss	4.836 ± 0.018
Residual MLP	MLP, normalized MSE	4.712 ± 0.006
Kernel-conditioned refiner	MLP on $(u, \mathrm{KRR}(u))$	4.730 ± 0.006
Fourier neural operator	spectral, metric loss, early-stopped	4.754 ± 0.031
Equal-weight ensemble	no fitting	4.648 ± 0.005
Fitted-weight ensemble	solved on validation	4.627 ± 0.006
Affine stack (per-pixel)	selected on a held-out half	4.595 ± 0.008
+ residual kernel correction	–	4.572 ± 0.010
Fourier neural operator	spectral, metric loss, complete schedule	4.682 ± 0.009
UNet	convolutional, metric loss	4.740 ± 0.030
Equal-weight ensemble, six members	no fitting	4.626 ± 0.003
Fitted-weight ensemble, six members	solved on validation	4.602 ± 0.003
Affine stack (per-pixel), six members	selected on a held-out half	4.567 ± 0.004
+ residual kernel correction	–	4.546 ± 0.003

said about the FNO, and adds the converse: a member that becomes more accurate and more like the others is worth nothing more.

The stages are more reproducible than the members they are built from, and the way the seed spread moves along the chain is itself informative: equal weighting, which fits nothing, takes the members' spread down to 0.005 points on test, and each subsequent fitted stage gives a little of that back — fitted global weights 0.006, the affine stack 0.008, the correction 0.010. The end of the chain is both the most accurate stage and the least reproducible one; fitting is not free, and a stack tuned harder is not automatically a stack that reproduces better (Supplement S6, Figure S6.6).

Every stage gain is small and every one is resolved. Pairing within each seed on the test set, equal weighting beats the best single member by 0.064 ± 0.007 points, fitted weights beat equal weighting by 0.021 ± 0.003, the affine stack beats fitted global weights by 0.032 ± 0.005, and the kernel correction beats the stack by 0.024 ± 0.005. All four improve at ten seeds out of ten, as does the end-to-end figure of 0.140 ± 0.014 points off the best member the pipeline contains. Each is a few hundredths of a point, which is precisely the resolution a single seed cannot deliver. The six-member chain resolves the same way: per-pixel weights beat the fitted global weights by 0.035 ± 0.003 points, the correction beats the stack by 0.021 ± 0.003, and the corrected pipeline sits 0.135 ± 0.008 points below its best member, each at ten seeds of ten.

Three selection rules were exercised rather than assumed, and all three are decided without touching the test block. Per-pixel affine weights are fitted on half the validation split and accepted only if they beat global convex weights on the other half: accepted at all ten seeds with five members, where with four they were declined at one. The correction is accepted on validation at all ten. And the member set itself is chosen the same way — validation prefers five members to four at ten seeds out of ten, which is what admits the FNO. The test-set consequence of that last decision, 0.039 ± 0.011 points, is read out afterwards and plays no part in making it. In the second campaign the per-pixel weights are accepted at all ten seeds with six members and the correction at all ten.

4.4 Which uncertainty is the binding one

Seeds are not the only source of uncertainty in Table 2, and on this benchmark they are not the largest. Every seed is scored on the same fixed 20000-sample test block, so the seed standard deviation measures training and selection variability and nothing else; the test block is itself a sample, and at a per-sample standard deviation of 0.0169 its contribution is 0.012 points, slightly larger than the pipeline's seed spread of 0.010. Treating the two as independent, the uncertainty on our reported 4.572% is about 0.016 points, and the seeds contribute a little under half of it. A study that replicates seeds and stops has measured the smaller half.

This decides what the table can support. Comparisons against a published number are unpaired, because we do not hold those methods' per-sample predictions, and so carry the test-block error twice: about 0.020 points in quadrature, and that figure imputes our own per-case variance to methods whose predictions, training variability and covariance with ours we do not know, so it is a sensitivity calculation and not a standard error of the difference. On it, the separation from PCA-Net, 0.098 points, is five times the sampling term, and the separation from PARA-Net, 0.022 points for the five-member pipeline and 0.004 for the six-member one, is at most one, and is therefore not a separation at all; we report the first as a benchmark ordering and the second as a tie. Comparisons inside the pipeline are paired, and pairing is worth far more here than seeds are: the end-to-end gain of 0.140 points is resolved at roughly a hundred standard errors at every seed, and the second campaign's 0.026 points over the first at about three. Supplement S6 works the budget through, including what it implies for the FNO's admission; the rule it leaves is the ordinary one — replicate seeds to learn whether a design choice survives retraining, pair on the test set to learn whether it helps, and never use either number to answer the other's question.

4.5 Architectural diversity and the shared residual

The natural reading of Table 2 is that the remaining error is a modeling problem: different architectures make different mistakes, so a more diverse ensemble should stack to a lower number. We tested this directly, and at every seed the measurements point the other way.

Figure S6.1 shows the correlation of per-sample normalized residuals between members. Over the ten seeds every pair of members correlates between 0.83 and 0.97, and the kernel predictor — the most alien member by construction, an exact solve on the raw loads rather than a trained network — still correlates 0.86–0.88 with the most accurate network. The mean pairwise correlation is 0.913 ± 0.005 across seeds. The second campaign adds a sixth

architecture and changes none of this: over its six members every pair correlates between 0.82 and 0.98, the mean is 0.919 ± 0.005, and the UNet correlates 0.91–0.96 with the networks it joins. The members are not making different mistakes. They are making the same mistake with small private variations, and how small does not depend on the seed or on the number of architectures.

The second-moment identity of Proposition 6.1 makes the consequence quantitative before any weights are fitted, and the campaign lets us check the prediction prospectively rather than illustrate it once. At each seed we measure the matrix S of normalized residual second moments on the validation split, solve for the optimal simplex weights, and compare the predicted root-mean-square relative error $\sqrt{w^\top S w}$ with what those weights realize. The prediction is $4.940 \pm 0.057\%$ and the realized mean relative error is a factor 0.938 ± 0.003 of it; on the six members of the second campaign the prediction is $4.920 \pm 0.057\%$ and the factor 0.938 ± 0.003. That factor is the dispersion between a root-mean-square and a mean, and its stability to three thousandths across ten independent replicates and two member sets is the substantive claim: S alone, measured without ever evaluating the metric, predicts the stack.

The weights it assigns are as informative as the risk it predicts. The optimum spreads across all five members at essentially every seed, the kernel taking a nonzero share at ten seeds out of ten and the plain MLP at nine; solved properly the answer is a spread, not a vertex. That the FNO takes the largest single share while ranking only third on the metric is the whole content of the diversity argument: S rewards a residual that is both small and unlike the others', and the spectral member is the only architecture here structurally unlike the residual-MLP family. The second campaign makes the point twice more. With a member that is not accurate: the optimum splits the weight three ways between the annealed FNO, the UNet and the refiner (0.30, 0.32 and 0.27), the UNet taking the largest share at four seeds of ten while ranking fifth of six on the metric, and removing it costs the pipeline 0.032 points. And with one that is: the annealed FNO is 0.072 points more accurate than its early-stopped checkpoint and worth nothing more to a stack of the same five members (Section 4.3), because its residual moved closer to the networks'. Part (ii) of the proposition puts the infinite-ensemble floor of an equicorrelated family at $\bar{e}\sqrt{\bar{\varrho}} = 4.922 \pm 0.056\%$ on five members and $4.910 \pm 0.057\%$ on six; that is a root-mean-square quantity and 4.572% is a mean, so the two are not on the same footing and we claim no margin below the floor (Remark 6.2). What the floor measures is how little the convex operation can buy, and the answer, at every seed and for either member set, is a few hundredths of a point. Supplement S6 carries the per-member weights, the pairwise admission tests, and the metric conversion.

Where does the common mistake live? Write r_m for member m's residual field on a validation case and $c = \frac{1}{M}\sum_m r_m$ for the across-member mean, the component that no amount of uniform averaging removes. Measured over the validation split, the shared component carries 90% of the average residual energy (Figure S6.2). Three of its properties matter. Spatially, its energy concentrates at the two corners of the loaded edge, where the traction boundary condition meets the lateral supports; relative to the local field scale it is three to four times larger there than the grid average, and these are precisely the locations where the finite element solution is least accurate and where interpolation to the 41×41 grid is most strained. Spectrally, it is enriched in high radial frequencies relative to the

stress fields themselves: the fields put 97.5% of their energy below radial wavenumber 4, the shared residual only 69%. And its magnitude is statistically independent of everything we can compute from the input: the correlation of $\|c\|$ with the output norm is 0.01, with the load norm 0.01, with the load's total variation 0.01. A component that no architecture avoids, that no input statistic predicts, that lives at the stress concentrations, and that fixed-scale additive noise would reproduce is consistent with noise of the data-generating process itself — finite element and grid-interpolation error at the singular corners — and harder to reconcile with approximation error of the surrogates. That is an inference, not a demonstration: every measurement here is made on surrogates. The experiment that would decide it is to re-solve a set of the hardest cases on a refined mesh or with higher-order elements and compare the solver discrepancy, case by case, with c; we have not run it, and the remaining measurements in this section should be read with that in mind.

Three further measurements are consistent with the same reading, and are reported in full in Supplement S6. The residual kernel correction improves the stack by 0.024 points at $n = 19000$ whatever the kernel scale, its tuner reaching for the smoothest kernel and the largest nugget on the grid: at this sample size the shared residual is not a function of the load in any way a Matérn RKHS on $\mathbb{R}^{41}$ can see. The response to data is small and, at ten seeds, resolved: under one protocol the normalized-MSE network's test error falls from $4.799 \pm 0.003\%$ at 8000 training samples to $4.712 \pm 0.004\%$ at 19000, a log-log slope of -0.022 ± 0.001 over that range against -0.052 for the kernel ridge, and the last 3000 samples buy 0.006 ± 0.004 points, paired over seeds. A doubling of the data, on either a power law or a floor fit to the ten-seed curve, is worth between four and eight hundredths, less than the stack and the correction buy at the present size. And the error does not respond to reseeding, at a scale that makes the comparison an argument: the five published architectures of Table 2 span 0.65 points, ten reseedings of our own pipeline 0.034, about five percent of it. A modeling limitation would be expected to yield to capacity, diversity, data or reseeding; within the range we could test, this error yields to each of them by hundredths, which is the pattern a floor in the data would produce.

The spatial structure of the remaining error — its concentration at the corners of the loaded edge, at every seed and in every member — is quantified in the online supplement (Section S6), together with the ablation of the pipeline's stages, the error spectra, and the cost accounting.

4.6 Seeds against architectures

The second campaign leaves sixty trained predictors on one test block: ten seeds of each of six architectures, the kernel ridge among them. Theorem 6.6 says what such a pool can reach in terms of three numbers, and the sixty let all three be measured on identical samples. The test block is split once into a calibration half of 1000 cases and an evaluation half of 19000, neither ever touched by training, checkpointing, stacking or tuning; weights are fitted on the first and every number below is read on the second.

The seed-to-seed correlation of one architecture is high: 0.92 to 0.99 across the six, 0.98 on average, against 0.83 to 0.98 between architectures, 0.92 on average. Reseeding therefore buys little. Averaging ten seeds of the FNO takes it from 4.679% to 4.649% and ten seeds of the normalized-MSE network from 4.709% to 4.680%; the metric-loss MLP, the least

reproducible member, gains the most, 0.16 points, and still ends above the others. Six architectures at a single seed, averaged with equal weights, reach $4.623 \pm 0.003\%$: one seed of each architecture beats ten seeds of the best one. All sixty together, at equal weights, reach 4.611%; with convex weights fitted on the calibration half, 4.586%; and the convex mixture chosen in hindsight on the evaluation half itself, which no rule can beat, reaches 4.581%, or 4.876% in the root-mean-square metric. The bound of Theorem 6.6, evaluated with the smallest member error and the smallest within- and between-architecture second moments of the pool, is 4.814% in that metric: no convex combination of these sixty can pass it, and the hindsight optimum sits six hundredths above it (Table 5).

Which architectures the pool cannot do without is a different ranking from which are best alone. Removing all ten seeds of one architecture and re-solving the hindsight optimum costs 0.021 points, root mean square, for the metric-loss MLP with reflection averaging, 0.018 for the UNet, 0.005 for the FNO and 0.001 for the kernel ridge, and nothing for the flat-loss and normalized-MSE networks, whose hindsight weights are zero; the mixture fitted on the calibration half moves by the same amounts within two thousandths of a point. The FNO is the best architecture on its own, at 4.937% over its ten seeds, and the reflection-averaged MLP the worst of the neural members at 4.979%, yet the pool misses the second four times more than the first. It is the neural member least correlated with the others, 0.92 with the UNet and 0.94 with the FNO, where the normalized-MSE network and the FNO correlate at 0.97; what a member is worth to a mixture is its decorrelation and not its own error, which is the second-moment identity read the other way.

Two readings follow. What a pool of trained members can reach on this benchmark is set by the correlation between architectures, and sixty members move it by hundredths; the deployed pipeline, six members at one seed with a per-pixel affine stack and a kernel correction, scores $4.543 \pm 0.003\%$ on the same evaluation half, below the hindsight optimum of every convex mixture of the sixty, because it is not a convex mixture; the same pipeline run over the ten-seed means of the six architectures, its stack and its correction fitted on the calibration half, reaches 4.533%, one hundredth lower, which is what Theorem 6.6 allows reseeding to buy once the architectures are fixed. And the hardest cases do not move at all: on the hardest one percent of the evaluation half, ranked by the best single member, that member scores 10.71%, its ten-seed average 10.66% and the sixty-member average 10.63%, against a median of 4.43%. Sixty predictors of six kinds agree on where the benchmark is hard, which is the shared residual of Section 4.5 seen from the largest pool we can assemble.

Three checks were run on the same split after the campaign. The first concerns the per-pixel stack over all sixty, which at the fixed ridge of the pipeline reads 4.682% against 4.556% for the six ten-seed means: the gap is the ridge, not the member count. With the ridge chosen by leave-one-out on the calibration half, the sixty read 4.547% and the six means 4.548%, and when the calibration half is cut to 300 or 120 rows the sixty stay ahead (4.563% against 4.593%, 4.581% against 4.672%). The second concerns the kernel member. A Matérn kernel with one length scale per load point, the scales learned on the training rows by the kernel flow of Owhadi and Yoo (2019) or by empirical Bayes and the overall size and nugget then set on validation, takes the kernel-ridge member from 5.19% on the evaluation half to 4.89% and 4.88% over ten seeds, where the isotropic kernel with the same validation grid reads 5.14% and the metric-loss MLP 4.83%; a programmed additive kernel, first-order modes plus a full mode with weights learned by the same flow, reaches 5.03%. Its

Table 5: Sixty predictors, ten seeds of six architectures, on the evaluation half of the test block (19000 cases). Mean relative L^2 error, and the root mean square where the theory speaks in that metric. Convex weights are fitted on the disjoint calibration half (1000 cases) unless marked hindsight, where they are chosen on the evaluation half itself. Rows with ± are means over the ten seeds.

Pool and weights	Error (%)	RMS (%)
FNO, one seed	4.679	4.967
FNO, ten seeds, equal weights	4.649	4.940
Six architectures, one seed, equal weights	4.623 ± 0.003	4.916 ± 0.003
Six architectures, one seed, convex, fitted	4.600 ± 0.003	–
Sixty, equal weights	4.611	4.905
Sixty, convex, fitted	4.586	4.880
Sixty, convex, hindsight	4.581	4.876
Lower bound of Theorem 6.6 on the sixty	–	4.814
Six ten-seed means, per-pixel affine, fitted	4.556	–
Sixty, per-pixel affine, leave-one-out ridge, fitted	4.547	–
Six means and the learned kernel, per-pixel affine, leave-one-out ridge	4.546	–
Six ten-seed means, per-pixel affine and kernel correction, fitted	4.533	–
Deployed six-member pipeline, one seed	4.543 ± 0.003	–

ridge can be chosen without a validation split: generalized cross-validation, the leave-one-out identity and the kernel alignment risk estimator select the same value as the validation rows at all ten seeds (5.473% at 8000-row fits). Added to the six means as a seventh, the learned kernel's ten-seed mean moves the stack from 4.556% to 4.555%, and from 4.548% to 4.546% with the leave-one-out ridge: a member that improves on the kernel by three tenths of a point is worth a thousandth to the pool, which is the decorrelation reading once more. The third is the kernel-flow regularizer of Section S1.2 on the high-data network, at ten seeds: the member reads 4.99% without it and 4.99% with it at weight 0.3, and 5.05% at weight 1.0 over five seeds.

4.7 Uncertainty quantification

Theorem 6.9 bounds the corrected surrogate's error by $\|G - m\|_K \, P_\lambda(u)$, and the finite-design diagnostic of the operator factor moves in the direction the theory suggests: with the deployed correction kernel, the fitted correction's squared RKHS norm is about 4900× smaller when the kernel regresses ensemble residuals than when it regresses raw stress fields, and about 8.8× smaller per unit energy of what is regressed (Table S6.2 in the supplement), the difference being the residual's smaller amplitude. Both numbers compare lower bounds on unknown norms at one nugget, so they describe the fitted objects rather than the smoothness of the residual operator. The design factor P_λ is nonetheless a poor pointwise indicator here. It correlates 0.08 with the corrected surrogate's absolute error and negatively with the benchmark's relative error, because it tracks the output norm (0.65) rather than the neural mean's mistakes, which are not a function of the input-space geometry P_λ sees. Ensemble

disagreement, the standard deep-ensemble signal, fails in the same direction and to the same degree (0.074 ± 0.006 against the absolute error, -0.25 ± 0.04 against the relative one), and adding the spectral member does not repair it (0.083 on four members): disagreement measures the member-specific error, which carries under a tenth of the residual energy, while the shared component is invisible to any within-ensemble statistic because the members agree precisely where they are jointly wrong (Section 4.5).

What holds is coverage, at every seed. The band the theory names, $q\,P_\lambda(u)$ with P_λ rebuilt from each seed's own correction kernel, is calibrated on 1000 cases drawn from the test block and evaluated on the disjoint 19000, neither used for training, checkpointing, stacking or kernel tuning, so exchangeability holds exactly (Section 6.5) and validity needs no correlation between score and error — which is as well, since the rank correlation between P_λ and the error is 0.03. For the six-member pipeline the observed coverage at the nominal 90% level is $89.9 \pm 0.8\%$, and the right comparison is with the exact finite-sample law rather than with 90%: the coverage of such a band is $\mathrm{Beta}(901, 100)$ distributed, whose central 95% interval is $[88.1\%, 91.8\%]$, and all ten seeds fall inside it; at the 95% level the interval is $[93.6\%, 96.3\%]$ and the observed coverage $95.0 \pm 0.6\%$, again ten of ten. A constant-width band calibrated the same way from the raw scores $\|e\|_2$ covers $90.4 \pm 0.8\%$ and $95.4 \pm 0.5\%$, also ten of ten at both levels, and the P_λ band is on average 1.11 times as wide as it: a scale factor that does not rank the error buys validity and nothing else. Rescaling the score by ensemble disagreement, which the correlations say should not help, indeed does not: $90.1 \pm 0.4\%$ and $95.0 \pm 0.8\%$, with nine seeds inside at the 95% level and the tenth on the band's upper edge, one excursion in the checks against central 95% bands, which is the rate the law itself predicts. The five-member pipeline of the same campaign repeats the pattern ($90.1 \pm 0.9\%$ and $95.1 \pm 0.8\%$ for the P_λ band, ten of ten and nine of ten inside). Supplement S6 carries the measurements behind each of these statements. The lesson is narrow but real: a valid pointwise bound is not automatically a useful pointwise indicator, two uncertainty signals of quite different provenance can both fail on the same problem, and a relative-error metric has to have its normalization accounted for before a posterior standard deviation means what it appears to.

5 The OCO-2 radiative-transfer emulator

The structural-mechanics benchmark put the neural mean and the kernel in the balanced regime. To see the other regime we apply the same components to the radiative-transfer emulation problem of Lamminpää et al. (2025): per spectral band of the OCO-2 instrument, learn the map from a reduced atmospheric state (20–24 dimensions after the dimension reduction of that paper) to the reduced radiance (40 PCA coefficients of the monochromatic spectrum). The OSF project (`u2t8a`) publishes a training pool of 20000 pairs and, separately, 2000 test states together with the kernel-flow emulator's own predictions on them; we verified the two sets share no point. We split the pool 18000/2000 into training and validation, and every choice in our pipeline — the network checkpoints, the kernel head's length scale and nugget, and the per-coordinate winners of the combination below — is made on that validation split; the 2000 public test states enter no optimizer and are evaluated once per run, for the final numbers. The comparison is therefore computed on identical test points against the published emulator itself rather than against our reimplementation of it.

Table 6: OCO-2 O2 band, scored on the 2000 public test states, which are disjoint from the training pool; all model selection happened on a validation split of the pool. Every trained row is mean ± standard deviation over ten seeds at a matched 250-epoch budget; the kernel-flow row is the emulator of Lamminpää et al. (2025), scored from its own stored predictions on fixed test points, and is identical at every seed. "Features" means the penultimate activations of the trained network; the kernel head is the same exact Matérn solve as everywhere else in this paper, and the two raw-input rows are fit by that same routine under the same budget as the feature heads: the same scale grid $\{0.5, 1, 2, 4\}$ times the median distance, the same nugget grid $\{10^{-8}, 10^{-6}, 10^{-4}\}$, the same 6000-sample tuning subsample, and the winner refit on all 18000 training points, which is why the selected hyperparameters vary from seed to seed. The ARD radiance cell is left empty because validation, which scores the reduced metric, ties between two hyperparameter configurations whose radiance differs by a factor of 2.2 (0.150% at seven seeds, 0.329% at three); a mean over that mixture would describe the selection noise, not the model. Table generated from the campaign artifacts by `campaign/gen_oco_table.py`.

model	reduced	radiance
Matérn kernel, raw input, one length scale	$40.57 \pm 0.10\%$	$0.233 \pm 0.003\%$
Matérn kernel, raw input, sensitivity-scaled (ARD)	$29.98 \pm 0.15\%$	—
kernel-flow emulator (Lamminpää et al., 2025)	16.89%	0.0448%
residual MLP, flat metric	$4.43 \pm 0.05\%$	$0.195 \pm 0.014\%$
residual MLP, weighted-coefficient loss	$49.32 \pm 0.40\%$	$0.0442 \pm 0.0044\%$
residual MLP, exact radiance loss	$59.02 \pm 0.76\%$	$0.0571 \pm 0.0057\%$
Matérn head on the flat network's features	$4.11 \pm 0.05\%$	$0.0793 \pm 0.0072\%$
Matérn head on the weighted network's features	$21.70 \pm 0.52\%$	$0.0319 \pm 0.0020\%$
Matérn head on the radiance network's features	$20.48 \pm 0.61\%$	$0.0379 \pm 0.0037\%$
per-coordinate combination	$\mathbf{4.12 \pm 0.05}\%$	$\mathbf{0.0294 \pm 0.0020}\%$

Two error metrics matter, and they disagree in an instructive way. The *reduced* metric is the relative L^2 error on the 40 standardized coefficients. The *radiance* metric maps predictions back to the monochromatic spectrum through the stored PCA projection and norms before comparing; because the projection is orthogonal, the error *numerator* on the reduced side is exactly the diagonally weighted norm $\|s_z \odot (\hat{z} - z)\|$, whose weights concentrate almost entirely on the first few coefficients (the denominator is the full reconstructed-radiance norm, which adds the reconstruction offsets). It is this diagonal weighting of the residual that the per-coordinate combination exploits.

Table 6 carries four findings, and Figure 2 shows the two central ones. First, the representation ladder: the same exact kernel machinery scores 40% on the raw input, 30% after rescaling each input coordinate by its measured sensitivity, and 4.1% on the trained network's features, past the network's own head. The raw-input kernel was limited by its features, not by anything kernel-shaped, and the deep kernel head, an exact solve on learned features, is the strongest single model on the reduced metric. Its margin over the network it feeds on ($4.11 \pm 0.05\%$ against $4.43 \pm 0.05\%$) is small — a third of a point — but it is not a seed accident: the head wins at ten of ten seeds on this band and at ten of ten on each of the

Table 7: Readout control for the kernel heads, ten seeds per band, mean ± standard deviation over seeds. For each trained network the same frozen last-layer features are read out three ways: by the network's own trained output layer, by a ridge regression refit on those features with the penalty chosen on validation, and by the exact Matérn head. The rows come from a rerun of the campaign script with the ridge rows added (`campaign/jpl_seeded.py`; body generated by `campaign/gen_oco_ridge_table.py`); the rerun's network and head rows reproduce Table 6 within the seed spread.

model	reduced	radiance
O2 band (10 seeds)		
network, flat loss	$4.43 \pm 0.05\%$	$0.195 \pm 0.014\%$
ridge readout on its frozen features	$4.71 \pm 0.07\%$	$0.228 \pm 0.014\%$
Matérn head on its frozen features	$4.11 \pm 0.05\%$	$0.0793 \pm 0.0072\%$
network, weighted-coefficient loss	$49.33 \pm 0.40\%$	$0.0439 \pm 0.0031\%$
ridge readout on its frozen features	$45.71 \pm 0.61\%$	$0.0521 \pm 0.0043\%$
Matérn head on its frozen features	$21.72 \pm 0.52\%$	$0.0316 \pm 0.0019\%$
WCO2 band (10 seeds)		
network, flat loss	$16.44 \pm 0.06\%$	$0.226 \pm 0.012\%$
ridge readout on its frozen features	$16.51 \pm 0.06\%$	$0.252 \pm 0.017\%$
Matérn head on its frozen features	$16.28 \pm 0.06\%$	$0.116 \pm 0.007\%$
network, weighted-coefficient loss	$62.57 \pm 0.56\%$	$0.0738 \pm 0.0057\%$
ridge readout on its frozen features	$61.32 \pm 0.43\%$	$0.0842 \pm 0.0067\%$
Matérn head on its frozen features	$48.67 \pm 0.45\%$	$0.0533 \pm 0.0054\%$
SCO2 band (10 seeds)		
network, flat loss	$8.23 \pm 0.02\%$	$0.199 \pm 0.015\%$
ridge readout on its frozen features	$8.33 \pm 0.03\%$	$0.236 \pm 0.032\%$
Matérn head on its frozen features	$8.08 \pm 0.01\%$	$0.109 \pm 0.007\%$
network, weighted-coefficient loss	$36.46 \pm 0.65\%$	$0.0751 \pm 0.0025\%$
ridge readout on its frozen features	$40.10 \pm 0.91\%$	$0.0866 \pm 0.0039\%$
Matérn head on its frozen features	$15.47 \pm 0.47\%$	$0.0631 \pm 0.0037\%$

other two, paired within seed. We still lean on the ladder's order of magnitude rather than on that gap. The gain is the kernel's and not a refitting of the readout: a ridge regression refit on the same frozen features (Table 7) is worse than the network's own trained output layer on the flat loss on every band at every seed, while the kernel head is better than both at every seed on every band, by 0.60 ± 0.07 points on O2 and about a quarter of a point on the other two; admitting the ridge readouts as candidates in the per-coordinate selection moves the combination by at most 0.002 points on any band.

Three statements of scope belong next to these numbers. The combination row is produced by the unweighted selector, which takes each reduced coordinate from the model with the smallest validation root-mean-square error on that coordinate; the campaign script also runs a selector weighted by the inverse squared norm of each validation state, the exact optimizer of the squared relative error that Proposition S2.2 covers, and it chooses different winners at three seeds on O2 and four on WCO2 yet lands on the same reduced and

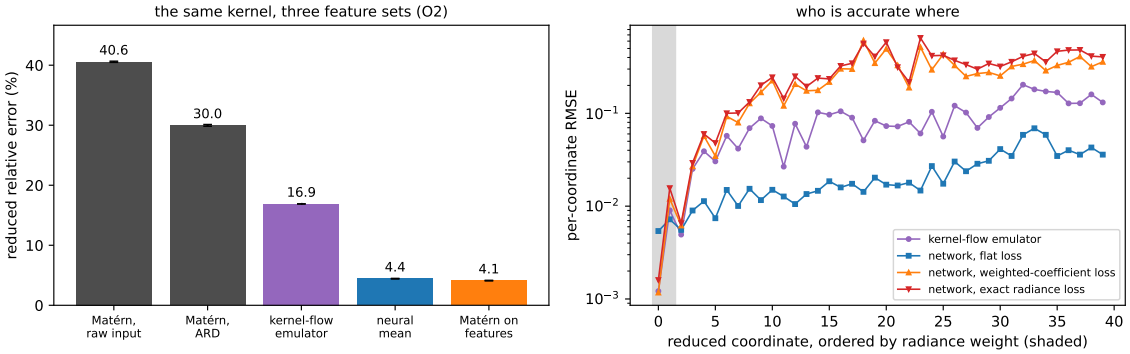

Figure 2: Both panels are drawn from the ten-seed campaign by `campaign/make_oco2_fig.py`: bars and error bars in the left panel are the mean and standard deviation over the ten seeds, curves in the right panel are per-coordinate test RMSE averaged over the same ten. Left: the representation ladder on the O2 band. The same exact Matérn machinery moves from 40% on the raw input to 4.1% on the trained network's features; the kernel-flow emulator and the network sit in between. Right: per-coordinate test RMSE with the reduced coordinates ordered by their radiance weight (shaded). Networks are named by the loss they were trained on. The per-output-tuned kernel-flow emulator is most accurate exactly on the heavily weighted leading coefficient (0.0012 against the flat network's 0.0054), while the flat network is nearly four times more accurate across the trailing thirty coordinates that the radiance barely sees (0.0276 against 0.1037). Both weighted losses buy the leading coefficient at the price of the tail, and by almost the same amount (0.0012 and 0.0016), which is the measurement behind the reading below; the per-coordinate combination takes each coordinate from whichever model wins it.

radiance means to the printed precision on every band (both records are in the release). The uncertainty of the comparison against the emulator is paired: on the same 2000 test states, a case-resampling bootstrap of the difference between the combination and the emulator excludes zero at ten of ten seeds on both metrics on O2 and SCO2, and at ten of ten on WCO2's reduced metric and nine of ten on its radiance metric, the exception being the seed that loses by 0.0015 points; the seed standard deviations in Table 6 describe retraining variability, not that comparison. And the claim is confined to what is rescored: the two criteria of Lamminpää et al. (2025) on the reduced coefficients and on the reconstructed monochromatic radiance, at a 250-epoch budget the networks have not converged at, with the emulator fitted on 20000 states against our 18000 plus a 2000-state validation carve. Instrument response, Jacobians and the retrieval itself are not evaluated, so nothing here says the pipeline is a better OCO-2 emulator end to end. Learning the metric rather than setting it by hand does not change this reading. A per-dimension metric fit by the kernel-flows cross-validation loss of Owhadi and Yoo (2019) reaches the same 30% on the raw state, and a low-rank Mahalanobis metric with a thousand more parameters does not improve on it; on the features, where the network has already conditioned the representation, a learned metric gives no gain over the isotropic kernel. The metric is a constant-factor lever, and the representation is the order-of-magnitude one.

5.1 What the metric buys, measured

The heuristic split of Proposition S2.5 — a constant σ_1^s always, a rate only under an approximate-rank gap — is scored directly by refitting both raw-input kernel rows at five nested training sizes, ten seeds per band, with the sensitivity map estimated causally at each size. The sweep, reported in full in the online supplement (Section S7), finds the constant and not the rate: the measured exponent difference is small on all three bands and consistent with zero on one, so a reader should take from the proposition only the part the measurement supports, and the representational term of Proposition 6.8 remains the order-of-magnitude lever.

5.2 Where the two-member rule can be trusted

Part (i) of Proposition 6.1 is the one result in this paper used as a decision rule rather than as a description: it says a weaker member helps exactly when $\varrho < e_1/e_2$, and we drop members on that test. A rule of that kind deserves to be scored the way it is used, so we decide it on the validation split and check it on the test block, for every pair of the eight members at every band-seed — $\binom{8}{2} = 28$ pairs at each of the thirty seeds, 840 in all.

Two separate things are being tested by this, and separating them is the point. The first is whether the proposition is true. Evaluated with both sides computed on the same block, the criterion $\varrho < e_1/e_2$ agrees with whether the optimal convex mix actually beats the better member on all 840 pairs, without exception. That is the proposition's own content rather than evidence about the world — it is an identity, and this is the numerical check that we implemented it correctly — but 840 instances is a strong form of that check, and it locates the difficulty elsewhere.

The second is whether the rule survives being used, which requires ϱ and e_1/e_2 to be estimated on data the decision does not see. Decided on validation and checked on test, the rule is correct on 62.7% of the 840 pairs. The average is the wrong summary: sorted by the margin $|\varrho - e_1/e_2|$ that the rule must resolve, the outcome is 105 of 255 below 0.02, 94 of 217 from 0.02 to 0.05, 79 of 86 from 0.05 to 0.10, 217 of 250 from 0.10 to 0.25, and 32 of 32 above 0.25 — that is 41%, 43%, 92%, 87% and 100%. These counts are not independent observations: the 28 pairs of a band-seed share members, and the ten seeds of a band share one test block, so the bins should be read as describing a trend across margin and not as binomial samples. Nothing about the proposition degrades in the small-margin pairs; what degrades is the transfer of two estimates from one block to another.

That transfer is measurable, and it is what sets the band. The signed gap $D = \varrho - e_1/e_2$ moves between validation and test with standard deviation 0.099 across the 840 pairs, essentially unbiased (mean -0.0005). This is an order of magnitude larger than the seed-to-seed scatter of D within a fixed pair, which is 0.010: replication across seeds is not what limits the rule, and a study that varied only the seed would have missed this. Because the pairs are binned by the margin the decision actually sees, it is part (ii) of Proposition S2.1 that speaks to the scoreboard, and it is a joint statement: if the estimation error were sub-Gaussian with variance proxy σ^2, the probability that a decision is both wrong and taken at an observed margin of at least 0.25 would be at most $\exp(-0.25^2/2\sigma^2)$, about four percent of all decisions, and the corresponding bound at a margin of 0.10 would exceed one half and say nothing. An empirical standard deviation over 840 dependent pairs does not

establish that proxy, so the four percent is an illustration of the proposition's scale and not a bound we claim. The proposition does not bound the error rate within a margin bin, which would require a lower bound on how often such margins occur, so the percentages above are an empirical calibration of the rule rather than a guarantee it carries. What they show is a trend consistent with the proposition: the rule's accuracy rises with the observed margin, and at the top of the range, where the joint bound leaves room for only a few percent of confident errors among the 840 decisions, the scoreboard records none in 32.

The practical reading, from the scoreboard rather than from the bound: on these problems the rule is dependable when the two quantities are separated by more than about 0.25, suggestive between 0.05 and 0.25, and uninformative below that. This applies to our own use of it. The kernel-versus-network decision earlier in this section has $\varrho = 0.14$ against $e_n/e_k = 0.109$, a margin of 0.031, which falls in the band the scoreboard says not to trust, and we therefore do not rest the exclusion of the raw kernel on that comparison. What does settle it is the same proposition read for magnitude rather than for sign: with these two errors and this correlation, part (i) puts the unconstrained two-member optimum at 4.4299% against the network's own 4.4320%, and the optimizing weight falls just outside the simplex, so no convex mix reaches even that value. Whichever side of the threshold the pair truly falls, the entire quantity in dispute is two thousandths of a point, so the member is dropped on the size of the available gain and not on a threshold the data cannot resolve. Reporting the margin alongside the verdict is what makes that distinction visible, and we recommend it wherever this rule is used.

5.3 Data scaling

The error on this problem is limited by data and yields to more of it; the measured scaling, and the surviving large-n observation on ClimSim after the withdrawal of the leaked small-n points, are in the online supplement (Section S7).

6 Theory

Throughout this section $\mathcal{U} = \mathbb{R}^p$ denotes the (discretized) input space and $\mathcal{V} = \mathbb{R}^q$ the output space, $G : \mathcal{U} \to \mathcal{V}$ the target operator, μ the input distribution, and $(u_1, v_1), \dots, (u_N, v_N)$ i.i.d. draws with $v_n = G(u_n)$; the data carry no sampling noise in the usual sense, coming from a deterministic solver, though Section 4.5 has something to say about solver error. We write $\ell(w, v) = \|w - v\|_2/\|v\|_2$ for the relative error and $R(\widehat{G}) = \mathbb{E}_{u\sim\mu}\, \ell(\widehat{G}(u), G(u))$ for the risk, which is the quantity reported in Section 4. The results below are largely elementary, but each one motivates a specific design decision in the pipeline of Section 3, and each has a measurable footprint in the experiments: one statement per stage, from the symmetry handling and the kernel-flow term through stacking, the residual correction, its coverage guarantee, and the choice of nugget. Proofs are collected in Supplement S8.

6.1 Guarantees for the fitted stages

Every fitted stage of the pipeline has an elementary finite-sample bound, stated and proved in the online supplement (Section S1). Their status should be read exactly: they are conditional capacity calculations for candidates fixed before the validation sample is drawn,

and the pipeline as deployed does not satisfy that hypothesis, because the same validation split that scores the stacks and the kernel grids also selects the network checkpoints, the member set and, on OCO-2, the coordinate winners. We therefore present them as the scaling behind each design decision, with constants measured on the runs, and not as guarantees for the shipped estimator; as Section S1 notes, several are numerically vacuous at the campaign's sizes in any case. The one statement that holds for the pipeline as deployed is the split-conformal coverage of Section 6.5, whose calibration split is drawn from the test block after every selection is fixed. The bounds are: a symmetrization inequality behind the reflection averaging (Proposition S1.1), the leave-half-out reading of the kernel-flow regularizer (Proposition S1.2), capacity bounds for the validated stack, the honesty comparison, the per-coordinate affine stack, and the grid-tuned correction (Propositions S1.3 and S1.5, Lemma S1.4, Corollary S1.6), the last with an analytic uniform bound on the correction weights (Lemma S1.7, $\kappa_{\mathrm{an}} = 500$, sharp over kernels and designs). All are capacity bounds, vacuous at the shipped constants and reported for their scaling; the statements whose evaluated value is the quantity reported are Theorems 6.9 and 6.10 below.

6.2 What the stack can achieve

Proposition S1.3 says fitting the weights is safe; it does not say mixing is worth anything. That is decided by a single measurable matrix. For a map f write the *normalized residual* at u as $\rho_f(u) = \big(f(u) - G(u)\big)/\|G(u)\|_2$, so that $\ell(f(u), G(u)) = \|\rho_f(u)\|_2$.

Proposition 6.1 (second-moment identity for convex stacks)**.** *Let $f_1, \dots, f_M$ be fixed maps and define $S \in \mathbb{R}^{M\times M}$ by $S_{mk} = \mathbb{E}_{u\sim\mu}\,\langle \rho_{f_m}(u), \rho_{f_k}(u)\rangle$. For w in the simplex Δ^{M-1} let $f_w = \sum_m w_m f_m$. Then*

$$\mathbb{E}_{u\sim\mu}\,\ell\big(f_w(u), G(u)\big)^2 \;=\; w^\top S\, w,$$

so the squared-metric risk of every convex stack is determined by S, and the best achievable is $\min_{w\in\Delta^{M-1}} w^\top S w$. In particular:

- *(i) (two members) if S has diagonal (e_1^2, e_2^2) with $e_1 \le e_2$ and correlation $\varrho = S_{12}/(e_1 e_2)$, mixing in the weaker member strictly helps if and only if $\varrho < e_1/e_2$, in which case the optimum is interior with value $e_1^2 e_2^2(1-\varrho^2)/(e_1^2 + e_2^2 - 2\varrho e_1 e_2)$;*
- *(ii) (equicorrelated family) if $S_{mm} = e^2$ and $S_{mk} = \varrho\, e^2$ for all $m \ne k$ with $\varrho \ge 0$, then $\min_w w^\top S w = e^2\big(\varrho + (1-\varrho)/M\big)$, attained at uniform weights and decreasing to $e^2\varrho$ as $M \to \infty$.*

Part (i) is the classical two-member minimum-variance rule, and part (ii) is the equicorrelated case of the ambiguity decomposition of ensemble learning (Krogh and Vedelsby, 1995); see Ueda and Nakano (1996) for the generalization-error form and Brown et al. (2005) for a survey. Both enter here as measurable decision tools on the estimated S.

Everything in this subsection is stated for the squared metric, and the tables of Sections 4 and 5 report a different one. The distinction is small numerically and decisive logically, so we make it explicit once and refer back to it wherever the two families of numbers meet.

Remark 6.2 (the two error metrics)**.** *For a map f write $\mathcal{E}_1(f) = \mathbb{E}\|\rho_f(u)\|_2$ and $\mathcal{E}_2(f) = (\mathbb{E}\|\rho_f(u)\|_2^2)^{1/2}$, the mean and the root mean square of the per-sample relative error. The*

tables report $\mathcal{E}_1$; Proposition 6.1 is a statement about $\mathcal{E}_2$, since $\mathcal{E}_2(f_w)^2 = w^\top S w$ by definition. Writing $X = \|\rho_f(u)\|_2$ and $c_f = (\operatorname{Var} X)^{1/2}/\mathbb{E}X$,

$$\mathcal{E}_2(f)^2 - \mathcal{E}_1(f)^2 = \operatorname{Var} X, \qquad \mathcal{E}_1(f)/\mathcal{E}_2(f) = (1 + c_f^2)^{-1/2} \le 1,$$

with equality only if the per-sample error is almost surely constant. Two consequences are used in this paper. The floor belongs in $\mathcal{E}_2$, because only $\mathcal{E}_2$ is determined by S: the map $w \mapsto w^\top S w$ is a quadratic form in second moments, whereas $w \mapsto \mathbb{E}\|\sum_m w_m \rho_{f_m}(u)\|_2$ depends on the full joint law of the residuals. And upper bounds transfer downward for free, since $\mathcal{E}_1 \le \mathcal{E}_2$, while lower bounds do not — so a floor read off S says nothing about a table entry without the dispersion of the same predictor. Comparing the two directly is biased in favour of the table entry by $(1 + c_f^2)^{-1/2}$, which is about six percent on the structural-mechanics stack and ten to eleven percent on the O2 objects of Section 5; the dispersion is a property of the predictor and not a constant of the problem, so a single conversion factor should not be reused across objects.

Part (i) is an exact criterion, but it is used as a decision rule and a decision rule is applied to estimates, so what governs it in practice is not the criterion but the gap it has to resolve relative to the noise in that estimation. A margin law for threshold rules of this shape (Proposition S2.1, Supplement S2) bounds the probability that a decision is both wrong and taken at an observed margin of at least τ by $\exp(-\tau^2/2\sigma^2)$, with σ the scale of the estimation error and not a chosen constant. That is a joint bound, not the error rate among the decisions whose observed margin exceeds τ: the conditional quantity is the joint one divided by $\mathbb{P}(|\widehat{D}| \ge \tau)$, on which the proposition says nothing. Accuracy inside a margin bin is therefore an empirical calibration of the rule, and Section 5.2 reports it that way.

Exact equicorrelation never holds in practice, but for convex weights the floor survives one-sided bounds on the entries of S, which are what one actually measures.

Corollary 6.3 (ensembling floor)**.** *If every member satisfies $S_{mm} \ge \bar{e}^{\,2}$ and every pair satisfies $S_{mk} \ge \bar{\varrho}\,\bar{e}^{\,2}$ with $\bar{\varrho} \in [0,1]$, then every convex combination f_w obeys*

$$\mathbb{E}\,\ell\big(f_w(u), G(u)\big)^2 \;=\; w^\top S w \;\ge\; \bar{e}^{\,2}\big(\bar{\varrho} + (1-\bar{\varrho})\,\|w\|_2^2\big) \;\ge\; \bar{e}^{\,2}\,\bar{\varrho}.$$

No number of additional members with the same error level and mutual correlations moves the ensemble's root-mean-square relative error below $\bar{e}\sqrt{\bar{\varrho}}$.

The floor is a lower bound; a matching upper bound follows from the same entrywise information, and together they pin the achievable window from both sides by measured quantities alone.

Corollary 6.4 (the floor is nearly achieved)**.** *Suppose, in addition to the hypotheses of Corollary 6.3, that $S_{mm} \le \bar{e}^{\,2}(1+\delta_e)$ for every m and $S_{mk} \le (\bar{\varrho} + \delta_\varrho)\,\bar{e}^{\,2}$ for every $m \ne k$. Then*

$$\bar{\varrho} \;\le\; \frac{1}{\bar{e}^{\,2}} \min_{w \in \Delta^{M-1}} w^\top S w \;\le\; \bar{\varrho} + \delta_\varrho + \frac{1 + \delta_e - \bar{\varrho} - \delta_\varrho}{M}.$$

The width of the bracket is the measured correlation spread δ_ϱ plus a $1/M$ term: when the members' errors and pairwise correlations are tight, whatever convex ensemble one builds sits within that window of the floor, and conversely no tuning of the weights can be blamed for more than the window.

On the structural-mechanics benchmark the bracket, evaluated with the one-sided bounds read off the measured S, is about a tenth of the squared floor wide, and the best convex mixture sits in its lower half at every seed: normalized by $\bar{e}^{\,2}$, the minimum is 0.980 ± 0.003 inside $[0.949, 1.039]$ on five members and 0.972 ± 0.005 inside $[0.933, 1.032]$ on six (Supplement S10). In the root-mean-square metric $\mathcal{E}_2$ the corollary is stated in, the mixture is at $4.940 \pm 0.057\%$ against a floor of $4.922 \pm 0.056\%$.

The floor is stated for convex weights, and the deployed stack is not convex: its per-pixel weights are affine and unconstrained in sign. For global weights the sign constraint can be removed exactly.

Corollary 6.5 (signed stacks)**.** *If S is positive definite, the minimum of $w^{\top} S w$ over all w with $\sum_m w_m = 1$, of either sign, is $1/(\mathbf{1}^{\top} S^{-1} \mathbf{1})$, attained at $w^{\star} = S^{-1}\mathbf{1}/(\mathbf{1}^{\top} S^{-1}\mathbf{1})$; it is at most the convex minimum, with equality exactly when $w^{\star}$ has no negative coordinate.*

On the measured S the two coincide at every seed and for both member sets: the unconstrained minimizer has no negative coordinate, so allowing signed global weights buys nothing here, and what the deployed per-pixel stack gains over the convex one (0.032 and 0.035 points in the two campaigns) is the variation of the weights over the grid, not their sign.

The members of a pool are usually of two kinds at once, architectures and seeds of each architecture, and the floor separates the two.

Theorem 6.6 (seeds and architectures)**.** *Let the pool consist of K seeds of each of M architectures, and suppose $S_{ii} \geq \bar{e}^{\,2}$ for every member, $S_{ij} \geq \varrho_w \bar{e}^{\,2}$ for two seeds of the same architecture and $S_{ij} \geq \varrho_b \bar{e}^{\,2}$ for members of different architectures, with $0 \leq \varrho_b \leq \varrho_w \leq 1$. Then every convex combination obeys*

$$\mathbb{E}\,\ell\big(f_w(u), G(u)\big)^2 \;\geq\; \bar{e}^{\,2}\Big(\varrho_b + \frac{\varrho_w - \varrho_b}{M} + \frac{1 - \varrho_w}{MK}\Big),$$

with equality at uniform weights when the three inequalities are equalities. In particular no number of seeds of one architecture takes the root-mean-square error below $\bar{e}\sqrt{\varrho_w}$, no number of seeds of M architectures takes it below $\bar{e}\sqrt{\varrho_b + (\varrho_w - \varrho_b)/M}$, and no pool of any size takes it below $\bar{e}\sqrt{\varrho_b}$.

Three measured quantities therefore set what any ensemble of trained members can reach: the member error, the seed-to-seed correlation of one architecture, which reseeding divides away, and the correlation between architectures, which it cannot touch. Section 4.6 measures all three on sixty predictors and compares the display with what the sixty achieve.

The two-member rule of Proposition 6.1(i) can be read the same way, as a statement about what a member's accuracy is worth against its correlation, and the exchange rate is explicit.

Proposition 6.7 (the price of accuracy in correlation)**.** *In the setting of Proposition 6.1(i) write $t = e_2/e_1$ and suppose the optimum is interior, $\varrho < \min(t, 1/t)$. The optimal two-member risk is $V = e_1^2\, t^2 (1 - \varrho^2)/(1 + t^2 - 2\varrho t)$, and*

$$\frac{\partial V}{\partial t} = \frac{2 e_1^2\, t\,(1 - \varrho^2)(1 - \varrho t)}{(1 + t^2 - 2\varrho t)^2} > 0, \qquad \frac{\partial V}{\partial \varrho} = \frac{2 e_1^2\, t^2 (t - \varrho)(1 - \varrho t)}{(1 + t^2 - 2\varrho t)^2} > 0.$$

A change $(\mathrm{d}t, \mathrm{d}\varrho)$ *in the second member therefore leaves the optimal risk unchanged to first order exactly when* $\mathrm{d}\varrho = -(1-\varrho^2)\,\mathrm{d}t/\big(t(t-\varrho)\big)$*: an improvement of the second member's error by a fraction* ϵ *is cancelled by a rise of* $\epsilon(1-\varrho^2)/(t-\varrho)$ *in its correlation with the first.*

Near the admission threshold, where $t - \varrho$ is small, the rate is large, and a member that becomes more accurate by converging toward the others can be worth nothing more to the stack; Section 4.3 reports the case, and Supplement S6 evaluates the display at the measured values.

A second consequence of the same decomposition explains the final OCO-2 surrogate. Evaluation metrics there are diagonal in the output coordinates (the radiance metric is the weighting s_z), and diagonal metrics decouple: the risk splits into a sum of per-coordinate risks, so the best combination is chosen coordinate by coordinate, and picking each coordinate's member on a validation split costs a uniform deviation term of the usual $\sqrt{\log(Mq/\delta)/m}$ size (Propositions S2.2 and S2.4 with Corollary S2.3, Supplement S2). This is what motivates the per-coordinate combination of Section 5. The propositions cover separable squared objectives with fixed coordinate weights; the two metrics the paper reports are means of relative norms, whose per-case denominators make them non-separable, so the theory does not make one selector simultaneously optimal for both. That the validation-selected combination improves on every single model on both metrics at once is an empirical finding, and Section 5 reports the selector it comes from and the weighted variant beside it.

6.3 Metric and representation: when a fixed kernel loses to a network

The structural-mechanics benchmark has the neural mean and the isotropic kernel within half a point of each other. On the OCO-2 emulation problem of Section 5 the same fixed kernel is an order of magnitude behind the same network. Two mechanisms can produce such a gap, and they separate cleanly.

The first is the input *metric*. If the operator factors as $G(u) = g(Au)$ with A nonsingular, and A has an approximate-rank gap at the sample size, a scattered-data calculation puts the isotropic rate at $\sigma_1^s n^{-s/d}$ against the adapted $n^{-s/r}$: a metric buys a rate under a rank gap, and the constant σ_1^s otherwise (Proposition S2.5, Supplement S2). The statement is labelled informal and nothing else in the paper rests on it. It holds under an idealization the supplement states, with no matching lower bound, and its rate half is not borne out here: refitting both raw-input kernel rows at nested training sizes over a sixteenfold range, at ten seeds per band, the measured difference between the two empirical exponents is small on all three bands and consistent with zero on one, although the sensitivity map's approximate rank is 0.54 to 0.60 of the input dimension — wide enough that the calculation would have permitted a rate gain. What the measurement does support is the constant: handing the kernel the anisotropic metric closes a factor of 1.1 to 1.5 out of a tenfold gap (Section 5.1). A reader should take from the proposition only that much. The second mechanism is *representational*: a stationary kernel of fixed smoothness reaches only its native space, at any metric, while the network adapts its features to the map, and no rescaling of the input can imitate that.

The representational term has a precise reading through the same optimal-recovery bound that Section 6.4 makes exact. That bound factors the error as $\|G - m\|_{\mathcal{H}} \cdot P_\lambda$, a product of the target's norm in the kernel's native space and a design factor set by the Gram

spectrum. Changing the features changes both in principle, and on the O2 band the two move by very different amounts. The effective dimension of the Matérn Gram, the quantity Lemma S5.1 controls, is nearly identical on the raw input and on the learned features (at $n = 4000$ and a matched median length scale, 3995 against 3993 at a 10^{-8} nugget); the design factor itself is not fixed by that trace, and measured directly at the 2000 test inputs the feature kernel's power function is smaller, its median 1.5 times below the raw kernel's. The fitted-norm diagnostic moves far more: regressing the same outputs through the two kernels at that setting, the squared native-space norm of the fitted interpolant, $\mathrm{tr}(\alpha^\top K\alpha)$, is 38 times smaller on the features than on the raw input. (An earlier draft quoted 42 from $\alpha^\top Y$, which adds the term $n\lambda\|\alpha\|^2$; at this nugget the two agree to three digits, and the network retrained for the check lands at 38.) Both numbers are finite-design diagnostics: the fitted norm is a lower bound on the unknown target norm, and the ratio depends on the kernel scale — 6 at half the median length scale, 154 at twice it — and collapses toward one at a 10^{-4} nugget, where the ridge no longer interpolates. Read as diagnostics, they say that the learned features hand the same kernel a target it interpolates with a much smaller coefficient norm, and a somewhat smaller pointwise design factor, which is the direction Proposition 6.8 predicts; they do not identify a causal decomposition into a fixed design factor and a moving norm.

This is not an accident of the O2 band; it is what a pulled-back kernel does. Writing φ for the feature map and k for the base kernel, the deep kernel head uses $k_\varphi(x, x') = k(\varphi(x), \varphi(x'))$, the construction studied as deep kernel learning (Wilson et al., 2016; Calandra et al., 2016).

Proposition 6.8 (feature pullback)**.** *The native space of k_φ is $\mathcal{H}_{k_\varphi} = \{f \circ \varphi : f \in \mathcal{H}_k\}$ with $\|g\|_{\mathcal{H}_{k_\varphi}} = \min\{\|f\|_{\mathcal{H}_k} : f \circ \varphi = g \text{ on } \mathcal{X}\}$. In particular, if the target factors through the features as $G = h \circ \varphi$ with $h \in \mathcal{H}_k$, then $\|G\|_{\mathcal{H}_{k_\varphi}} \le \|h\|_{\mathcal{H}_k}$, and the optimal-recovery bound of Theorem 6.9 for the feature-kernel regression is governed by $\|h\|_{\mathcal{H}_k}$ rather than by the norm of G in the raw-input space.*

The proof (Supplement S8) is the standard pullback identity for reproducing kernels (Saitoh, 1997; Paulsen and Raghupathi, 2016). Its content here is the reading: a fixed kernel on the raw input must carry the whole warped map G, whose native-space norm is large when G has fine structure; the same kernel on the features carries only the unwarped h, and training drives φ toward exactly the factorization that makes h simple. The measured factor of 38 in the fitted norm is the finite-design shadow of that gap, and it is the representational advantage that an anisotropic metric, which rescales the input but cannot re-express the map, leaves untouched (recovering only the factor 1.4).

6.4 An error bound for the residual kernel correction

The final stage of the pipeline is kernel ridge regression of the ensemble residual. The bound below is a vector-valued, residual form of the classical power-function estimate for kernel interpolation (Wendland, 2004) and of the optimal-recovery viewpoint of Owhadi and Scovel (2019); we include the short argument in Supplement S8 to keep the two factors explicit. Fix the mean $m : \mathcal{U} \to \mathcal{V}$ (in the pipeline, the stacked ensemble; in the statement below m is any fixed map independent of the correction sample) and write $r = G - m$ for the residual operator with components r_j, $j = 1, \dots, q$. Let k be a positive definite kernel on $\mathcal{U}$ with

RKHS $\mathcal{H}_k$, and assume $r_j \in \mathcal{H}_k$ for all j with

$$\|r\|_K^2 := \sum_{j=1}^{q} \|r_j\|_{\mathcal{H}_k}^2 < \infty.$$

Given inputs $X = (u_1, \ldots, u_n)$, Gram matrix $K = k(X, X)$ and nugget $\lambda \geq 0$, the correction is $\widehat{r}_\lambda(u) = k(u, X)\,(K + n\lambda I)^{-1} R$, $R_{nj} = r_j(u_n)$, and the corrected predictor is $m + \widehat{r}_\lambda$. Let

$$P_\lambda(u)^2 \;=\; k(u,u) - k(u,X)\,(K + n\lambda I)^{-1} k(X,u) \tag{1}$$

denote the Gaussian process posterior variance with the same nugget (for $\lambda = 0$ this is the classical power function of the point set X).

Theorem 6.9 (conditional power-function bound)**.** *For every $u \in \mathcal{U}$ and every $\lambda \geq 0$,*

$$\big\|G(u) - m(u) - \widehat{r}_\lambda(u)\big\|_2 \;\leq\; \|G - m\|_K\, \widetilde{P}_\lambda(u) \;\leq\; \|G - m\|_K\, P_\lambda(u),$$

where $\widetilde{P}_\lambda(u)^2 = P_\lambda(u)^2 - n\lambda\,\|(K + n\lambda I)^{-1} k(X,u)\|_2^2$. The first inequality is sharp: for every u there is a residual operator of norm $\|G - m\|_K$ at which it holds with equality, so $\widetilde{P}_\lambda(u)$ is the exact worst-case error of the correction over the ball $\{r : \|r\|_K \leq \|G - m\|_K\}$.

Three comments. First, the inequality is algebraic: it holds pathwise for every fixed m, every dataset, and every u, with no probabilistic assumptions. In the pipeline m is itself fit on the same training set; this does not affect the validity of the bound, only the reading of $\|G - m\|_K$ as a random (realized) quantity rather than an a priori one. Second, the bound factorizes into a quantity that depends only on the residual operator ($\|G - m\|_K$) and a quantity that depends only on the kernel and the design (P_λ), and the second factor is exactly the posterior standard deviation that a Gaussian process interpretation of the correction would report. The bound is conditional in two ways that the reader should keep in view: it presumes that every coordinate of the residual $G - m$ lies in the Matérn native space, which is not established for a finite-element target on $\mathbb{R}^{41}$, and its constant $\|G - m\|_K$ is unknown — the fitted norms of Table S6.2 are lower bounds on it, not estimates — so the theorem does not deliver a numerical error bar. What it delivers is the shape of the worst case: the error at u is $P_\lambda(u)$ times a constant that does not depend on u. We stress that this is a statement about the form of a valid upper bound, not about the usefulness of P_λ as a pointwise error *ranking*. Section 4.7 finds that on this benchmark P_λ ranks the error poorly, because the neural mean supplies most of the error and the relative metric is confounded by output amplitude; a distribution-free conformal rescaling of P_λ nonetheless attains its nominal coverage on the test set. Third, the theorem quantifies why one should regress residuals rather than raw targets: the design factor P_λ is the same in both cases, so the gain of the neural mean is the drop from $\|G - \bar{v}\|_K$ (kernel-only, mean $\bar{v}$) to $\|G - m\|_K$. These norms are not directly observable, but the RKHS norms of the *fitted* interpolants, $\|\widehat{r}_\lambda\|_K^2 = \mathrm{tr}(\alpha^\top K \alpha)$ with $\alpha = (K + n\lambda I)^{-1} R$, are computable lower-bound proxies, and they drop by a factor of about 4900 when the kernel is moved from raw targets to ensemble residuals (Table S6.2). That ratio is a finite-design diagnostic and no more: it compares two lower bounds, and much of it is amplitude, since the residual is already a few percent of the target; normalized by the energy of what each kernel regresses, the drop is about 8.8

rather than 4900 (Table S6.2). It says that the fitted residual interpolant has a much smaller diagnostic norm, not that the residual operator is smoother in the native-space sense.

The design factor cannot be improved by a better estimator either. Among all rules that see the same training residuals, the interpolant is optimal in the worst case over the ball, and the price of a positive nugget in that worst case is explicit.

Theorem 6.10 (minimax optimality over the native-space ball)**.** *Fix a design X with K positive definite, a point u and a radius $\rho > 0$. For every map $\Psi : \mathbb{R}^{n\times q} \to \mathbb{R}^q$ that estimates $r(u)$ from the training residuals $r(X)$,*

$$\sup_{\|r\|_K \le \rho} \big\| r(u) - \Psi\big(r(X)\big) \big\|_2 \;\ge\; \rho\, P_0(u),$$

and the kernel interpolant $\Psi(R) = k(u, X)K^{-1}R$ attains the bound. For $\lambda > 0$ the worst case of the ridge correction over the same ball is $\rho\,\widetilde{P}_\lambda(u)$, and, with $k_u = k(X, u)$ and $A_\lambda = K + n\lambda I$,

$$P_0(u)^2 \;\le\; \widetilde{P}_\lambda(u)^2 \;\le\; P_\lambda(u)^2, \qquad \widetilde{P}_\lambda(u)^2 - P_0(u)^2 = (n\lambda)^2\, k_u^\top A_\lambda^{-2} K^{-1} k_u,$$

so the nugget's cost in the worst-case constant is a computable quantity that vanishes as $\lambda \to 0$.

The lower bound is the classical optimal-recovery argument (Owhadi and Scovel, 2019): two residuals of norm ρ that vanish on the design and take opposite values at u are indistinguishable from the data, so any rule errs by at least $\rho P_0(u)$ on one of them. What the theorem adds for the pipeline is a reading of the reported quantity: at $\lambda = 0$ the worst-case constant $\|G - m\|_K\, P_0(u)$ over the native-space ball is the smallest any rule can attain from these data, and at the validated nugget the gap to it is the displayed correction term, evaluable from the Gram matrix alone. The statement is about the worst case over a ball whose radius is not known; it is not an operational error bar for the shipped surrogate, which is the split-conformal set of the next subsection.

6.5 Distribution-free coverage for the reported band

Theorem 6.9 controls the error by $\|G - m\|_K\, P_\lambda(u)$, but the constant $\|G - m\|_K$ is not known a priori and Section 4.7 shows that P_λ alone ranks the error weakly. What the surrogate reports as uncertainty is therefore a rescaling $q\, P_\lambda(u)$, whose multiplier is the split-conformal quantile of the scores $\|e(\tilde{u}_i)\|_2 / P_\lambda(\tilde{u}_i)$ on a calibration split of the test block, drawn after every selection in the pipeline has been made on the validation split; Section 4.7 reports this band beside a constant-width band calibrated the same way from the raw scores $\|e(\tilde{u}_i)\|_2$. Coverage needs neither the bound to be tight nor P_λ to rank the error: it needs exchangeability alone, which the protocol provides, and it then holds in finite samples, $1 - \alpha \le \mathbb{P}(G(u^\star) \in C(u^\star)) \le 1 - \alpha + 1/(m + 1)$ (Proposition S3.1). The realized coverage has an exact law as well — $\mathrm{Beta}(k, m + 1 - k)$ at $k = \lceil (1 - \alpha)(m + 1) \rceil$, whence the $[88.1\%, 91.8\%]$ band against which Section 4.7 places the ten seeds (Remark S3.2). Both statements, with their proofs and the comparison of this band with the constant-width and disagreement-scaled bands calibrated on the same split, are in Supplement S3.

7 Discussion

Two regimes, one pairing. The two problems bracket what a kernel stage after a neural mean can do. On structural mechanics the families tie, and the kernel corrects the residual of a stacked mean; the improvement over the published band decomposes into pieces that ten replicates can size. Accurate neural means trained on the reported metric and averaged over the reflection symmetry do most of the work, from the kernel's 5.197% to the best single network's 4.712%. Combining members adds what their measured residual correlations permit and no more — 0.064 points from equal weighting, 0.021 from weights fitted on validation, 0.032 from letting those weights vary over the grid — and the kernel correction contributes 0.024 ± 0.005 points, bringing with it the residual bound of Theorem 6.9 and the coverage of Proposition S3.1. Each contribution is smaller than the difference between two published architectures, and only the seeds make them separable from noise. The largest single lever is which members the stack contains: removing the spectral member costs 0.039 ± 0.011 points, more than any stage above, although it ranks third of five on the metric. What a member is worth to a stack is set by how its errors covary with the others', not by how small they are — the second campaign's FNO, annealed to the end of its schedule, is 0.072 points more accurate and worth nothing more, while a UNet that ranks fifth of six is worth 0.032 — and a stack of near-duplicates cannot be repaired by tuning the combination harder. Sixty of them, ten seeds of six architectures, reach 4.611% at equal weights and 4.581% at the best convex weights chosen in hindsight, a floor Theorem 6.6 places from three measured correlations (Section 4.6).

On OCO-2 the kernel on the raw state trails the network by an order of magnitude, a stack gains nothing measurable from it, and its value moves inside the network as an exact head on the learned features; the pipeline's job becomes metric management, training each member in a loss that concentrates where the reported metric concentrates and combining per coordinate. In both regimes the same two measurements decide the design before any stack is fit: the pairwise residual correlations, which set the ensembling floor of Corollary 6.3 and, through Corollary 6.4, bracket how close the fitted ensemble comes to it, and the member error ratio, which the second-moment identity turns into a keep-or-drop verdict for each candidate.

Against PARA-Net the five-member pipeline is level, 0.022 points or about one test-block sampling term, and the six-member pipeline of the second campaign is level too, 0.004 points the other way; we read both as a tie. Without the spectral member the pipeline trails by 0.061 points, about three such terms, so the tie belongs to the configurations that carry that member. The margin over the other published methods is a benchmark ordering rather than a tested difference, for the reason Section 4.4 gives. These are the last few learnable hundredths above the shared-residual plateau that Section 4.5 measures for the predictor pool we trained — a plateau consistent with a floor in the benchmark data, and not shown to be one — not a step on the way to zero: a sixth architecture moves the pipeline by 0.032 points, training the FNO to the end of its schedule moves the member by 0.072 points and the pipeline not at all, and the residual correlations that set the floor do not move either.

Relation to prior work. Neither component of the pairing is new. Mora et al. (2025) place a neural operator inside the mean of a Gaussian process and fit both jointly; deep kernel learning (Wilson et al., 2016; Calandra et al., 2016) puts a kernel on a network's

features and trains through it; and the kernel-flow line of emulation work, of which the OCO-2 baseline of Lamminpää et al. (2025) is the closest instance, has shown that kernels with data-adapted hyperparameters replace physics codes within measurement precision once the input is in its physical parametrization and the target has been reduced to something smooth. The differences here are operational, but they matter at this scale: the mean is fit first and the kernel after, so the expensive stage is ordinary network training; the kernel is tuned by validation rather than by marginal likelihood; the correction is an exact solve at $n = 19000$ rather than a training-time approximation; and the feature-space stage is evaluated against the same kernel on the raw input under the same budget, which is what lets the representation effect be measured (Table S6.2 and Section 5). The theory of Section 6 applies verbatim to the jointly trained setting; the measured drop in the fitted RKHS norm is the quantitative reason residual corrections of accurate means are the right place to spend kernel capacity. The step this paper does not take is the retrieval-level one. The OCO-2 Level 1 and Level 2 products are publicly distributed through NASA's Earthdata archive, and evaluating how the surrogate's conformal bands and fallback behavior propagate through an actual retrieval, rather than through the emulation metrics alone, is the natural continuation.

Limitations. The benchmark has a one-dimensional input function and a fixed geometry; the exact kernel solve exploits both, and problems with high-dimensional input fields would need the usual approximations (inducing points, random features), which give up the exact solve and with it the residual bound of Theorem 6.9 in the form stated. That bound controls the error relative to the RKHS norm of the residual operator, a quantity we can only lower-bound empirically; the capacity bounds for the stacks and the correction have measured constants but crude ones, and neither they nor the bound should be read as a pointwise error bar — the conformal band is the only object here that plays that role. Training the means on the metric, the reflection steps and the stacking are benchmark-agnostic, but the error levels reported here are properties of these datasets and protocols.

The two structural-mechanics campaigns differ in the FNO's schedule, and the difference is measured rather than assumed: the early-stopped member of the first campaign trails the annealed member of the second by 0.072 points at every seed (Section 4.3), and the stage-by-stage accounting is reported for both. The OCO-2 networks are trained at a matched 250-epoch budget short of convergence; a 750-epoch run reaches lower levels on the O2 band without changing the orderings (Supplement S7). Our low-data protocol reproduces that of Mora et al. (2025) up to the ambiguity of which 1250 samples are used; we use the first 1250 of the training block and the same 20000-sample test set, and release the splits.

The reading of the structural-mechanics plateau as a data floor is an inference from surrogates alone. The experiment that would decide it is to re-solve a set of the hardest cases on a refined mesh or with higher-order elements and compare the solver discrepancy with the shared residual of Section 4.5; we have not run it. Proposition S2.5 is an informal rate calculation rather than a theorem with a matching lower bound, and the OCO-2 sweep supports only its constant half: at a rank ratio between 0.54 and 0.60 it would permit a rate gain from an anisotropic metric, and across a sixteenfold range of training sizes at ten seeds the metric delivers a constant between 1.1 and 1.5, with no rate gain on one band and small ones on the other two (Section 5.1).

Outlook. Three directions seem worth pursuing. First, the benchmark itself: the re-solve described above, or regenerated data with refined meshes at the corner singularities, would settle whether the plateau is a property of the data and, if it is, return the problem to being a test of surrogates. The diagnostic used here — cross-family residual correlation, the second-moment prediction of Proposition 6.1 and the scaling-in-n check — is cheap to run on any benchmark suspected of the same condition. Second, the same recipe on problems where the input functions are genuinely high-dimensional, where the interplay between neural means and kernel corrections should look different and the exact solve will not be available. Third, the uncertainty side: P_λ is a design quantity, so it can be optimized, and the connection of Lemma S5.1 between the nugget, the spectrum and the effective dimension suggests principled ways to spend a fixed computational budget on the correction stage.

Acknowledgments and funding

The research presented in this paper was supported by the European Research Council (ERC) under the European Union's Horizon 2022 research and innovation programme (grant agreement No. 101041711), by the Simons Foundation as part of the Collaboration on the Mathematical and Scientific Foundations of Deep Learning, by Heights Labs, by Convex Nexus Capital, by the Israel Science Foundation (grant number 2258/19), and by the Israel Science Foundation (ISF Grant 4101/25).

Declaration of competing interest

The author declares no competing interests.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author made substantial use of generative AI tools, and describes their role here. A large part of the software was written with Fable 5 (Anthropic): the implementation of the experiments and diagnostics, the downloading, assembly and preparation of the benchmark and OCO-2 datasets, and the adaptation of the publicly released emulator code of Lamminpää et al. — originally split between Julia and a ReFRACtor-driving Python layer — into a single Python pipeline for sampling states and reducing radiances. The main contributions are the author's: the residual coupling of neural means and exact kernel corrections, the two-regime framing that organizes the paper, and the supporting theory, all developed with AI assistance in calculation, drafting, and exposition. Each theorem and its proof was checked independently and more than once — by Fable 5, by ChatGPT Pro (OpenAI), and by the author — and the reported numbers were verified against the released per-run data. The author reviewed all of the above and takes full responsibility for the content of the manuscript.

Data and code availability

The code, the per-run summaries behind every reported number, and the exact configuration of each experiment are available at `https://github.com/yspennstate/neural-means-kernel-corrections`. The structural-mechanics, Helmholtz, Navier–Stokes and advection data are distributed through the data record accompanying de Hoop et al. (2022) (`data.caltech.edu`, record `20091`); the OCO-2 emulation data and the kernel-flow emulator's predictions through the OSF project `u2t8a`; and the ClimSim data through the LEAP repository `subsampled_low_res`.

Supplementary material

An online supplement accompanies this paper, referenced throughout as Supplement S1–S10. It is available at `https://github.com/yspennstate/neural-means-kernel-corrections/blob/main/paper/supplement.pdf`, in the repository that holds the code, and is built from the same sources as this manuscript. It contains the finite-sample bounds for every fitted stage and all proofs, the margin and per-coordinate selection results, the effective-dimension identity, the ablation, error-localization, spectra, uncertainty-quantification and cost detail on structural mechanics, the uncontrolled survey of the remaining suite problems and the mirror-symmetry check, the OCO-2 input-metric study with the data-scaling and ClimSim results, the implementation and compute record, and the numerical verification of every stated result. The main text is self-contained: the supplement carries derivations and measurements, not claims that are made only there.

References

Jinho Baik, Gérard Ben Arous, and Sandrine Péché. Phase transition of the largest eigenvalue for nonnull complex sample covariance matrices. *Annals of Probability*, 33(5):1643–1697, 2005.

Pau Batlle, Matthieu Darcy, Bamdad Hosseini, and Houman Owhadi. Kernel methods are competitive for operator learning. *Journal of Computational Physics*, 496:112549, 2024.

Gavin Brown, Jeremy Wyatt, Rachel Harris, and Xin Yao. Diversity creation methods: a survey and categorisation. *Information Fusion*, 6(1):5–20, 2005.

Roberto Calandra, Jan Peters, Carl Edward Rasmussen, and Marc Peter Deisenroth. Manifold Gaussian processes for regression. In *International Joint Conference on Neural Networks (IJCNN)*, pages 3338–3345, 2016.

Andrea Caponnetto and Ernesto De Vito. Optimal rates for the regularized least-squares algorithm. *Foundations of Computational Mathematics*, 7(3):331–368, 2007.

Matthieu Darcy, Boumediene Hamzi, Giulia Livieri, Houman Owhadi, and Peyman Tavallali. One-shot learning of stochastic differential equations with data adapted kernels. *Physica D: Nonlinear Phenomena*, 444:133583, 2023.

Maarten V. de Hoop, Daniel Zhengyu Huang, Elizabeth Qian, and Andrew M. Stuart. The cost-accuracy trade-off in operator learning with neural networks. *Journal of Machine Learning*, 1(3):299–341, 2022.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

Martin Drohmann and Kevin Carlberg. The ROMES method for statistical modeling of reduced-order-model error. *SIAM/ASA Journal on Uncertainty Quantification*, 3(1): 116–145, 2015.

Marc C. Kennedy and Anthony O'Hagan. Predicting the output from a complex computer code when fast approximations are available. *Biometrika*, 87(1):1–13, 2000.

Vladimir Koltchinskii and Evarist Giné. Random matrix approximation of spectra of integral operators. *Bernoulli*, 6(1):113–167, 2000.

Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: learning maps between function spaces with applications to PDEs. *Journal of Machine Learning Research*, 24 (89):1–97, 2023.

Anders Krogh and Jesper Vedelsby. Neural network ensembles, cross validation, and active learning. In *Advances in Neural Information Processing Systems 7*, pages 231–238. MIT Press, 1995.

Sawan Kumar, Rajdip Nayek, and Souvik Chakraborty. Neural operator induced Gaussian process framework for probabilistic solution of parametric partial differential equations, 2024. arXiv preprint.

Otto Lamminpää, Jouni Susiluoto, Jonathan Hobbs, James McDuffie, Amy Braverman, and Houman Owhadi. Forward model emulator for atmospheric radiative transfer using Gaussian processes and cross validation. *Atmospheric Measurement Techniques*, 18: 673–694, 2025.

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.

Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.

Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*, 393:114778, 2022.

Ziqi Ma, David Pitt, Kamyar Azizzadenesheli, and Anima Anandkumar. Calibrated uncertainty quantification for operator learning via conformal prediction. *Transactions on Machine Learning Research*, 2024. arXiv:2402.01960.

Emilia Magnani, Marvin Pförtner, Tobias Weber, and Philipp Hennig. Linearization turns neural operators into function-valued Gaussian processes. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. arXiv:2406.05072.

Vladimir A. Marčenko and Leonid A. Pastur. Distribution of eigenvalues for some sets of random matrices. *Mathematics of the USSR-Sbornik*, 1(4):457–483, 1967.

Carlos Mora, Amin Yousefpour, Shirin Hosseinmardi, Houman Owhadi, and Ramin Bostanabad. Operator learning with Gaussian processes. *Computer Methods in Applied Mechanics and Engineering*, 434:117581, 2025.

Sebastian W. Ober, Carl E. Rasmussen, and Mark van der Wilk. The promises and pitfalls of deep kernel learning. In *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence*, 2021.

Houman Owhadi and Clint Scovel. *Operator-Adapted Wavelets, Fast Solvers, and Numerical Homogenization*. Cambridge University Press, 2019.

Houman Owhadi and Gene Ryan Yoo. Kernel flows: from learning kernels from data into the abyss. *Journal of Computational Physics*, 389:22–47, 2019.

Vern I. Paulsen and Mrinal Raghupathi. *An Introduction to the Theory of Reproducing Kernel Hilbert Spaces*. Cambridge University Press, 2016.

Sai Prasanth, Ziad S. Haddad, Jouni Susiluoto, Amy J. Braverman, Houman Owhadi, Boumediene Hamzi, Svetla M. Hristova-Veleva, and Joseph Turk. Kernel flows to infer the structure of convective storms from satellite passive microwave observations. AGU Fall Meeting Abstracts, abstract A55F-1445, 2021.

Saburou Saitoh. *Integral Transforms, Reproducing Kernels and their Applications*. Longman, 1997.

Naonori Ueda and Ryohei Nakano. Generalization error of ensemble estimators. In *Proceedings of the IEEE International Conference on Neural Networks*, volume 1, pages 90–95, 1996.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Oriol Vendrell-Gallart, Nima Negarandeh, and Ramin Bostanabad. Geometry-aware post-hoc uncertainty quantification in operator learning, 2026. arXiv:2606.17513.

Holger Wendland. *Scattered Data Approximation*. Cambridge University Press, 2004.

Andrew Gordon Wilson, Zhiting Hu, Ruslan Salakhutdinov, and Eric P. Xing. Deep kernel learning. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*, volume 51 of *Proceedings of Machine Learning Research*, pages 370–378, 2016.

David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

Gene Ryan Yoo and Houman Owhadi. Deep regularization and direct training of the inner layers of neural networks with kernel flows. *Physica D: Nonlinear Phenomena*, 426:132952, 2021.